

DETECTION OF MENTAL STATE OF A HUMAN USING EEG SIGNALS

by

Sriparvathi Shaji Bhattathiri

Graduate Paper Submitted

in

Partial Fulfillment

of the

Requirements for the Degree of

MASTER OF SCIENCE

in

ELECTRICAL ENGINEERING

Approved by:

Dr. Ferat Sahin, Professor, Research Advisor

Dr. Sohail A. Dianat, Professor, Department Head

DEPARTMENT OF ELECTRICAL AND MICROELECTRONIC ENGINEERING

KATE GLEASON COLLEGE OF ENGINEERING

ROCHESTER INSTITUTE OF TECHNOLOGY

ROCHESTER, NEW YORK

April 2020

Graduate Paper

Detection of the Mental State of a Human Using EEG Signal

By Sriparvathi Shaji Bhattathiri

Ms. Electrical Engineering

March 2020

Plagiarism Declaration

This report has been written by me to present the work I did for my graduate paper. With the exception of any statements that have been referenced, all the material presented in this report is the result of my own efforts. Any code that has been borrowed has also been properly credited. I understand that any evidence of plagiarism and/or the use of unacknowledged third party materials will be dealt with as a serious matter.

Signed

Sripavathi Shaji Bhattathiri

Acknowledgements

I thank my advisor Dr Ferat Sahin, for his guidance and support without which this project would not have been successful. I would also like to thank all the members of the MABL Lab for providing insight and help whenever I needed it. I would also like to express my humble gratitude towards RIT for bestowing upon me the opportunity to work on such a project. Last but not the least, I would also like to thank the Electrical Engineering department, friends and everyone who helped and motivated me.

Abstract

With advancement in the field of technology, it will be of immense value to be able to understand the emotional state of a human being especially during human robot interactions. The emotions of a human can be detected from various physiological signals of which EEG signals are used in this research. The goal of the paper is to detect if a human has positive, negative or neutral emotion during a time span from the recorded EEG signals. Various machine learning algorithms like support vector machine (SVM), K nearest neighbour (KNN), Random Forest (RF) and Adaboost is used to classify the emotions. The best technique is chosen.

Contents

1	Introduction	1
2	Literature Survey	3
2.1	Preprocessing	4
2.2	Feature extraction,Feature Selection and Classification	5
2.3	Data set	9
3	Proposed Methodology	12
3.1	Preprocessing	12
3.2	Feature extraction	12
3.2.1	Statistical Features	12
3.2.2	Spectral Features	17
3.2.3	Event Related Potential	18
3.3	Feature Selection	19
3.4	Classification	20
3.4.1	SVM	21
3.4.2	KNN	21
3.4.3	Random Forest	22
3.4.4	Ada-Boost	23
3.4.5	Artificial Neural Networks	23
4	Results	25
5	Conclusion	27
A	MATLAB Code	33

A.1	Preprocessing and Feature Extraction main	33
A.1.1	Session 1	33
A.1.2	Session 2	42
A.1.3	Session 3	52
A.2	Feature Extraction Functions	62
A.2.1	Statistical Features	62
A.2.2	Spectral Features	65
A.2.3	More Statistical Features and Event Related Potential	66
A.2.4	Discrete Wavelet Features	68
A.3	Saving Data	73
B	Python Code	75
B.1	Feature Reduction and Feature Selection	75
B.1.1	PCA	75
B.1.2	Random Forest for feature selection	75
B.2	Classification	77
B.2.1	SVM	77
B.2.2	KNN	78
B.2.3	Random Forest	79
B.2.4	Adaboost	83
B.2.5	Neural Network	83

List of Figures

2.1	Basic Emotions on the Valence Arousal Dimensional Model [1]	4
3.1	Number of principle components to represent 99% of the data	19
3.2	A plot of features against its importance	20
3.3	Code, training and testing accuracy of a single SVM classifier	21
3.4	Training and testing accuracy of ensemble KNN	21
3.5	Training and testing accuracy of ensemble KNN	22
3.6	A single estimator from the Random Forest	22
3.7	Random Forest	23
3.8	Ada-boost architecture and performance	23
3.9	Training and testing accuracy of Adaboost with PCA	23
3.10	Test error as the number of trees in Adaboost increases	24
3.11	Architecture of the artificial neural network	24
3.12	Accuracy obtained	24

List of Tables

4.1	Comparison of accuracy of the best model using each algorithm	25
-----	---	----

Chapter 1

Introduction

With the advancements in the field of robotics, robots are more accurate, efficient and safe enough to freely interact with humans under various circumstances and are not constraint to do repetitive task in caged work-spaces. This makes it imperative that humans feel comfortable around robots. The first step in ensuring ease of interaction is to understand the emotional state of a human during the course of interaction. If this can be achieved, robots can indeed become an integral part of our every day life. It is important to understand the human emotion during interaction with an industrial robot which is faster and more powerful than a more innocuous seeming small robot. This stems from the fact that industrial robots are required to do the challenging tasks that we hope to delegate to robots. Hence, this paper concentrates on detection of emotional state and specifically the comfort of a human is detected using Electroencephalogram signals (EEG). EEG signals as proposed by Hans Berger are electrical signals from the brain which can be a direct indicator of the emotional state. They are more advantageous than methods like fMRI, MEG and provides a non-invasive and cost effective method which has almost no health hazard. [2] These signals recorded by non-invasive methods reflect the electrical activity in the cortex of the brain. The intensity of these signals are rather small and measure in micro-volts.

The EEG signals usually fall in the frequency ranges know as alpha (8Hz-13Hz), beta (lower (14Hz-21Hz) and upper (21Hz-30Hz)) and gamma (30Hz-40Hz) and theta (4Hz-7Hz). [3] [2] These signals have been widely used to understand intention in human computer interface and though lesser many researchers are also using EEG signals to have robots/computers with a certain level of emotional intelligence.

This rest of the project is organised as chapter II which gives insight into related works, chapter III that explains the data set used, chapter IV presents the proposed methodology while chapter V shows the results and VI briefly discusses the conclusions drawn as well as the future scope.

Chapter 2

Literature Survey

EEG signals vary based on tasks and is very subject specific and hence models have to be trained for each specific task [4] but EEG signals have been successfully used by various researchers in human robot interactions to predict the intent behind actions, emotions and much more. [5]

When classifying the emotional state of a human, two different models are usually used, one is categorical which consists of a finite number of discrete states, each representing a specific affect category or dimensional models which are continuous models and represents varying intensities of affects. The categorical model usually defines happiness, sadness, fear, anger, disgust, surprise and neutral as the states of emotion and is frequently used in human robot interaction. The clear distinction between the states is its strength while the lack of ability to classify any affect that is not included in these categories is its major drawback. The dimensional models were developed to overcome the problems pertaining to classifying affects across continuous spectrum. The dimensional models either use two or three dimensions. An example would be a three dimensional model which takes into consideration, pleasure vs. displeasure, arousal vs. calm and relaxation vs. tension. Similar models may have components like attention vs. reject etc. An example of two dimensional model would include positive activation-negative activation model to account for high arousal and neutral valence. The two dimensional model is also used in human robot interaction. [6] Valence represents while arousal represents activation level. The former goes from positive to negative and the latter ranges from low to high. [1] The six emotions plotted on a valence-arousal graph is as shown below in Fig.3.10. Sometimes psychological issues can also play a role in human-robot interaction. [7] Due to various factors many thus hold the belief that the robot human interaction should be understood more holistically and not constrained to

emotional states understood using physiological signals and subjective evaluation. [8] In spite of the above mentioned drawbacks, EEG has been widely used in classifying emotions. [9] [10] [11] Further majority of research 64.91% of papers survey by Al-Nafjan et al. used a dimensional model while 12.83% of papers used a discrete model and the rest used a different model or did not specify what model was used. Hence, in this paper using EEG to understand the emotional state of a human during human robot interaction will be explored further. [11] The

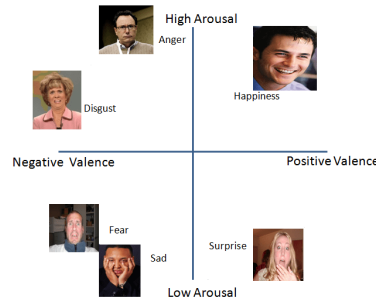


Figure 2.1: Basic Emotions on the Valence Arousal Dimensional Model [1]

following subsections deal with previous work pertaining to emotion detection using EEG signals specifically the pre-processing, removing noise, feature selection, dimensionality reduction and the most suitable algorithms to be used to classify EEG signals.

2.1 Preprocessing

Authors almost always try to avoid artifacts as much as possible but they can still occur. Hence, it is always a good practice to perform artifact removal. In his survey paper Alarcao et al. observed that 84% of works used band pass filters to filter and retain the alpha, beta and gamma components of EEG signals for emotion detection. [11] This also removes the DC offset of the electrodes as well as the noise from power line at 50 Hz and even drifts caused by electrode impedance. [12]

There are some common methods like Principle Component Analysis (PCA) which linearly transforms data to retain only the orthogonal components, Independent Component Analysis (ICA) that separates the various components of the data (artifacts can then be identified and removed), Surface Laplacian (SL) which filters out those signals that originated outside the skull hence emphasising on electrical activity spatially close to a recording electrode, etc. used for removing non-brain signals. [10] [13] The SL filter also attenuates the EEG signal hence

improving the spatial resolution of the signal that is recorded but poses the danger of eliminating useful information contained in the middle frequencies. [14] Static Filters, Spatial Filters, Adaptive filtering etc. are also used for removing artifacts like muscle tension, room temperature, environmental noise etc.. [2]

In his survey paper Alarcao et al. observed that about 19.3 % of work used Blind Source Separation (BSS) and 8.8 % used ICA. This was done to remove blinking, eye movements, line noise etc. It is also observed that referencing of electrodes using Average Mean Referencing (AMR), Common Average Reference (CAR) and Laplacian was rather common. [11] A combination of the above methods will prove to be useful in artifact removal depending upon the artifacts present in the collected data.

2.2 Feature extraction, Feature Selection and Classification

EEG signals possess many frequency domain as well as time domain features along with some frequency-time domain features. Alarcao et al. found that Short-time Fourier Transform (STFT) or Discrete Fourier Transform (DFT), Statistical features, Power Spectral Density (PSD) features, Wavelet Transform features, Entropy, Higher Order Crossing (HOC), Common Spatial Pattern, Fractal Dimensions (FD) and Asymmetry Index are some methods of feature extraction in decreasing order of popularity. It was also observed that 89.4% of all studies surveyed used only alpha, beta, delta, gamma and theta bands and a 37% of them used the above mentioned bands in conjunction [11]

PSD uses Fast Fourier Transform (FFT) to compute the Discrete Fourier Transforms (DFT) and its inverse. Each signal can hence be decomposed into component frequency ranges. For emotion detection, alpha, beta, theta and gamma are usually of interest. Delta does not provide much information. In frequency domain features, power spectral density is widely used in feature extraction. [15] [16] [3] Though found in a lot of studies, the performance score of frequency domain features, specifically power spectral density is lower than that of some other features. [3]

Discrete wavelet transform (DWT) is an alternate to power spectral density. It has an advantage over power spectral density that it retains the time domain information as it has good temporal localization. [3] [15] This is achieved by using a non-zero wavelet kernel that is moved over all the points in the original EEG signal. This is a time limited sine wave. Such time-frequency analy-

sis is usually done using convolution. [2] The DWT decomposes the signals into approximation co-efficients (CA) and detailed co-efficients (CD) iteratively. [17] [13]

Liu et al. in their paper use time domain features like statistical features (power, mean, standard deviation, first difference, normalized first difference, second difference, normalized second difference, normalized difference, HOC, FD, Hjorth feature, non-stationarity index (NSI) along with PSD features, DWT features (Band energy, recursive energy efficiency, root mean square) and some multi-electrode features (differential asymmetry, rational asymmetry, magnitude-squared coherence estimate, mutual information and Pearson correlation co-efficient). Then feature extraction techniques were applied to get the best subset of feature. [3] This is a common practice. In this paper, Maximum Relavance Minimum Redundancy (mRMR) and Principal Componenet Analysis (PCA) were applied and was used for classification of arousal and valence with little more 71% accuracy using Random Forest (RF) and slightly lesser using SVM . [3]

Similar approach is used by Ackermann and Kohlschein et al. in their paper as the EEG data is decomposed into alpha, beta, gamma and theta bands and HOC as well as Hilbert-Huang Spectrum (HHS) features are obtained along with STFT features(minimum, maximum, variance), power spectral density features (maximum, minimum, variance, power averaged over time and alpha/beta-power ratio) and mean instantaneous frequency(obtained by applying hilbert function to Intrinsic Mode Functions(IMFs) and averaging). Then mRMR is used in its Mutual Information Difference version to select the best features and used in classification with RF which out-performs SVM. [18]

In [16] an approach similar to [18] and [3] is used where time domain features (statistical like power, mean, standard deviation, first difference, second difference along with HOC, FD, Hjorth features) as well as PSD, DWT and some multi-electrode features like differential asymmetry and rational asymmetry, magnitude squared coherence estimate are extracted. Then mRMR is applied for feature selection. Using RF a classification accuracy of about 70% is obtained while classifying arousal and valence, while K-nearest neighbour (KNN) does not perform as well. [16] Hence it can be easily observed that it is a common practice to extract multiple features and use feature selection techniques to find the best subset of features for a particular problem.

In [19] PSD, signal power, CSP, univariant statistics were used. When single feature is used for classification, PSD gives the best accuracy (83%) with Linear Discriminant Analysis (LDA). When using various combination of the features, combination of CSP, PSD, spectral power gave an accuracy of 88% with LDA out-performing SVM and multilayer perceptrons.

In the work presented in [20] dimensionality reduction is performed by reducing the number of input channels based on a fitness function which chooses channels with highest correlation with class label and lowest correlation with other channels. The feature extraction is performed after this wherein statistical features like mean, standard deviation, skewness and kurtosis are obtained and only feature values of the useful channels are used. With LDA as the classifier an accuracy of 82% is obtained which is substantially better than previous models. [20] In [14] also reduced set of channels are used and the performance is compared to using all channels and the performance was evaluated on KNNs and LDA. The set of features used were obtained after performing wavelet transform and included standard deviation, variance, power and entropy. KNNs outperformed LDA by giving an accuracy of 83% in classification while using all 62 channels and 79.19% while using reduced channels. [14]

Ang, Yeong and Ser in their paper extract two time domain features, namely, mean and standard deviation along with two frequency domain feature i.e. power of the signal and maximum amplitude of the frequency components specifically arising from left frontal and right frontal as well as DWT features (Daubechies order 4 and Symlets order 6). It is observed that DWT individually gives the highest accuracy (77.3%) while classifying with an ANN model. Among models that use combination of features, the one using maximum frequency amplitude ratio, power, DWT performed best with an accuracy of 81.8%. [17] In the paper [21] PSD features were used with neural networks to obtain 70% accuracy in classification of emotional state. The subject involved both healthy individuals as well as patients. [21]

Other works include feature selection using empirical mode decomposition [22] as well as using specific features like in [14] in which Murugappan, Ramachandran and Sazalia use DWT and derive statistical features and an energy based feature called Recoursing Eneregy Efficiency (REE) in two different forms, Logarithmic REE (LREE) and Absolute Logarithmic REE (ALREE). The energy based features gave an accuracy of 83.26% performing better than other features when used with KNN and 79.93% when used with LDA. [13]

Feature Engineering is among the most challenging aspects of EEG processing while using traditional machine learning algorithms. Most of the research pertaining to use of deep learning to process EEG signals hope to get rid of the hassle of feature engineering by employing deep neural networks (DNN) for automatic feature learning. [23] About 49% of papers reviewed by Roy and Banville et al. used raw EEG data as input and 48% used hand engineered features while 3% used other inputs. It was also observed by them that DNNs can be applied to raw EEG signals

to achieve state-of-the-art results. Of the papers that used extracted features time frequency domain representation of EEG using Short-time Fourier transform was common. Delta, theta, alpha, beta bands were widely used for mental state detection. Other features used involved, PSD features, entropy, kurtosis, fractal components etc. [2]

For emotion detection from EEG signals using deep neural network, Al-Nafjan et al. used PSD, frontal EEG asymmetry, mean absolute errors and pearson correlation were used. PSD was used to obtain the average power over a specified frequency range and this acted as a feature. Frontal EEG asymmetry studies alpha and beta frequency range which is consistently linked to emotional states. These can be used to detect the arousal and valency which in turn can be used to predict the emotion in conjunction with other features. It is also observed that DNN outperforms RF. [15]

Hence it can be safely concluded that the best starting point for research would be to obtain DWT features, common frequency domain and time domain features and perform mRMR or PCA to obtain the best subset. It also shows random forest and deep learning would be the best choice for classification of EEG signals.

Support Vector Machines(SVM)

In [11] it has been stated that almost 59% of all papers they surveyed used SVM with different kernels to classify EEG signals to detect emotions. The radial basis function is followed in popularity by linear, polynomial, Gaussian kernels. Further variations of SVM like least square SVM and adaptive SVM were used. Hence it can be easily concluded that SVM be a good technique to start with and would provide baseline model to compare the performance of other models with.

Random Forest

Random forest developed by Leo Breiman, is a technique based on decision trees. Each node corresponds to a feature and branch to the values taken by the feature. At each branch the feature with the highest information gain is selected using entropy. [16] As observed in the earlier subsection, it performs better than most classical machine learning algorithms. The efficiency stems from it being an ensemble model. It uses multiple learning methods and hence improves accuracy. Trees are usually grown fully and then pruned. [24] Some papers, like [24] boasts of an efficiency of almost 97% obtained in the classification of emotions. Hence Random

forest seems like a good choice of machine learning technique to be used with EEG signals.

Deep Learning

In [23] it has been stated that of a total of 102 studies analysed, deep learning provides an average gain of 5.4% in comparison to baseline machine learning algorithms and in some cases even lead to 76.7% increase in accuracy. While deep learning is a broad term applied to many neural network models, some of the different architectures commonly used as observed by Roy and Banville et al. in their survey of paper presented in [23] is Convolutional Neural Networks (CNNs) constitute 41%, Recurrent Neural Networks (RNNs) about 14%, Auto Encoders (AEs) almost 13%, combination of CNNs and RNNs almost 7% and the rest is constitute of all other architectures. CNNs learn filter banks and share weights. They are good at learning spatial information. RNNs have intra-layer connections and are efficient in learning temporal information. [25] Hence , it is a safe bet to start with a CNN architecture and decide the number of layers and loss function depending on the performance of the model. Further using RNN with CNN will be advantageous in classification of EEG as both spatial and temporal information will be captured by such architectures.

2.3 Data set

There are a wide variety of studies conducted on emotion detected using EEG signals as discussed earlier. They use different number of electrodes and varying electrode positions. [11] reports that most studies use between 14 to 64 electrodes with an average of 52 electrodes. The most prominent electrode positioning system is 10-20 and was applied to 32.14% of all studies surveyed in [11], with the average number of electrodes being 41 and ranging between 1 to 64 electrodes while the 10-10 system has been used by 5.4% of the work and always used 64 electrodes. A lot of studies use DEAP dataset or similar experimental setup like the one used by them. [20] [16] [18] [22] [15] [3] This involves recording a base-line neutral EEG pattern. Then eliciting emotions using a video, followed by watching a black screen and a break. Then the process is repeated. This is a common frame work followed by other studies also where the participant is first made to relax followed by giving a trigger and then allowing the stimulus to die out and then taking a break. An example would be [19] in which pictures were used to elicit emotions. Each trial was 5s of being exposed to the picture stimulus and 5s of interval for 20

pictures. There were 12 participants. Sometimes audio-visual stimulus are used but only the method of eliciting emotions changes, rest of the procedure is similar. [12] The time duration for which the participant is exposed to the stimulus, the focus time, the interval and the survey time varies. Ex. focus time may be 2s followed by 4s of stimulus and a survey time of 8s which also acts as an interval in [17]. Many at times the experimental set-up is in a dark room. [17] There are studies in which EEG from pictures and sound clips (lasting 76s and an interval of 16s) are used as stimulus separately and EEG signals are recorded. [26] Almost all of these surveys were conducted on healthy adults though some were conducted on patients with some abnormalities like in [21]. in [11] it has been observed that average duration of 11.9s while the actual length varies between 1.5s of stimulus to 48s. To make sure that the results obtained are comparable to some of the standard data-sets as well as other experiments, the emotional stimulus should be about 11.9s and preceded by a focus time of 2s and an about 8s to fill out the survey which would also act as an interval.

In this particular project, a standard data set has been used, namely, emotions eNTERFACE06.EMOBRAIN data set. The data set contains physiological signals from galvanic skin response, respiration, and blood volume pressure and EEG and fNIRS nervous systems. For this project only the EEG signals are used.

There are three labels attached to the data, which pertain to the emotion experienced categorised as neutral, positive and negative. The data set consists of EEG collected from 5 participants over 3 different session. During each session, the participants were exposed to visual stimulus which elicited one of the three emotions. There were 30 such stimulus encompassed of videos. It is noteworthy that Session 1 of participant 1 was recorded at a lower frequency 256 Hz as compared to other sessions recorded at 1024 Hz. The stimulus for Participant 1 and 2 for the first session was not considered adequate and the stimulus was changed. The sessions lasted 15 minutes each. Every stimulus was separated from the next with a 10s break while the stimulus lasted for about 12.5s. [27]

This data set has been used by studies both as bench mark as well as to demonstrate their feature extraction and feature selection methods. Some of the papers that have used this data set include the work done by Anne-Claire et al. which suggests the use of features such as spectral moments (spectral centroid, spectral width, spectral asymmetry and spectral flatness) along with heuristic spectral shape descriptors like spectral crest factors, spectral slope, spectral decrease, spectral variation as well as common spatial pattern along with common time domain

and frequency domain features. The paper uses 54 channels. They followed leave one block out protocol for testing and reports accuracy for each model used for predicting the emotion of each of the subjects. In other words the models are all subject dependent and gives a maximum accuracy of 73% for 2 class classification of their emotions. Inter-session as well as intra-session accuracy varies. Some subject data shows better results in intra-session while others show better results in inter-session. [28]

Further [28] also reports discrepancies where the image label which is the ground truth corresponding to that image stimulus is different from the subjects' self assessment. Hence predicting the emotion for a particular participant may be more difficult than that of another.

While [29] extracts features from EEG signals from the same data set and uses feed forward neural network as well as multi-layer neural network with two hidden layers. The classification model for each subject is trained independently of the other.

Yet another paper [30] uses the same data set but only the EEG signals from 3 participants, uses naive bayes classifier to obtain 60% accuracy for two classes. The features used were event related potential, Short Time Fourier Transform features, power spectral density and a combination of these.

[31] has done work very similar to that aimed by this project. Three emotional states were classified for all participants using EEG signals. The extracted features included both frequency domain features as well as time domain features. A 51% accuracy was obtained using KNNs. LDA gave 51% accuracy. The classification accuracy increased to about 67% when predicting just two classes.

classifying just two emotions instead of 3 was done with 56% using Bayes and 57% using artificial neural networks by [32].

Chapter 3

Proposed Methodology

This section is organized as sub-section A which explains the pre-processing followed by B giving details pertaining to feature extraction and feature selection. The last subsection deals with the machine-learning algorithms used for classification.

3.1 Preprocessing

The artifacts are minimised during data collection by providing a guideline to the participants which includes requesting minimal head movements and blinking. Further band pass filter is used to obtain just the alpha, beta and gamma frequency ranges as they provide the vital information required for emotion classification.

3.2 Feature extraction

A number of different features are extracted and feature selection applied to obtain the best subset. The features extracted can be broadly classified as statistical features, spectral features as well as event related potential features. The statistical features can be extracted in the time domain, frequency domain as well as of the power spectral density. The same applies for the spectral features.

3.2.1 Statistical Features

The statistical features are as follows-

- Mean

It is one of the most commonly used statistical feature. It gives the average value within the data set. The formula for the same is as follows-

$$Mean = \bar{X} = \sum_{i=1}^N X_i / N \quad (3.1)$$

where X_i refers to the i th instant and N refers to the total number of instances in the data set. [33] There is a MATLAB function `mean()` available for calculating mean.

- Standard deviation

Often abbreviated as STD, it is another commonly used statistical feature. It represents the dispersion of a data set from its mean value. It can be represented as follows-

$$STD = \sqrt{\sum_{i=1}^N (X_i - \bar{X})^2 / N - 1} \quad (3.2)$$

where X_i refers to the i th instant and N refers to the total number of instances in the data set and $\bar{X}$ refers to the mean of the data set as explained by Equation 3.1. [33] There is a MATLAB function `std()` available for calculating standard deviation.

- Energy

It is a measure of the strength of a signal. Hence it can be calculated as follows-

$$Energy = \sum_{i=1}^N (X_i) \quad (3.3)$$

where X_i refers to the i th instant and N refers to the total number of instances in the data set. [33]

- Co-efficient of Variation (COV) This is the ratio of standard deviation to mean of the data distribution where mean is as obtained by Equation 3.1 and standard deviation as obtained by Equation 3.2 [33]

- Minimum

This gives the lowest value present in the particular data set. It is a value of statistical

importance especially when dealing with bio-signals. There is a MATLAB function `min()` which performs the operation.

- Maximum

Maximum refers to the maximum value present in the data set. This is also a good statistical feature when dealing with biological signals. In this particular case it refers to the highest voltage recorded.

- Variance

It is a power index and is defined as the square of standard deviation. It is represented by the following equation-

$$\sigma = STD^2 = \sum_{i=1}^N (X_i - \bar{X})^2 / N - 1 \quad (3.4)$$

where X_i refers to the i th instant and N refers to the total number of instances in the data set and $\bar{X}$ refers to the mean of the data set as explained by Equation 3.1. [34] There is a MATLAB function `var()` available for calculating variance.

- Root mean square value

Popularly abbreviated as RMS value, it a good measure of a signal. It can be defined as

$$RMS = \sqrt{\sum_{i=1}^N (X_i)^2 / N} \quad (3.5)$$

- Willison Amplitude

Abbreviated as WAMP, Willison Amplitude is used as a measure of frequency information. It takes into consideration a threshold and gives 1 if the value is above the threshold and 0 otherwise.

- Log Detector

Log detector is a feature that stores the value as defined by-

$$LOG = e^{1/N \sum_{i=1}^N (\log|X_i|)} \quad (3.6)$$

- Skewness

It is a measure of asymmetry in the data. It is often quantified as how much the given distribution varies from normal distribution. It is represented as -

$$Skewness = \frac{(\sum_{i=1}^N (X_i - \bar{X})^3 / N)}{\sqrt{\sum_{i=1}^N (X_i - \bar{X})^2 / N}} \quad (3.7)$$

This calculation may be have system bias and can be corrected by multiplying with a factor of $\sqrt{n-1}/(n-2)$ It can be calculated using MATLAB function skewness()

- Kurtosis

It can be described as the sharpness of a distribution or the tailness of a distribution. It is representative of how much the data is prone to outliers.

$$Kurtosis = \frac{(\sum_{i=1}^N (X_i - \bar{X})^4 / N)}{\sqrt{\sum_{i=1}^N (X_i - \bar{X})^2 / N}} \quad (3.8)$$

There is a MATLAB function called kurtosis() which can be used for its calculation. Like skewness, kurtosis is also prone to systemic errors and this can be corrected by performing the following where K_0 represents the corrected kurtosis and k_1 represented the kurtosis with the error -

$$k_0 = \frac{(n-1)}{(n-2)(n-3)} ((n+1)k_1 - 3(n-1)) + 3 \quad (3.9)$$

- First Difference

It can be expressed with the following formula-

$$\delta = \frac{1}{T-1} \sum_{t=1}^{T-1} |S(t+1) - S(t)| \quad (3.10)$$

Here $S(t)$ represents an instant of the sample at time t and is similar to X_i in the previous equations. [16]

- Normalized first difference

It is the normalised first difference represented as -

$$\bar{\delta} = \frac{(\frac{1}{T-1} \sum_{t=1}^{T-1} |S(t+1) - S(t)|)}{\sigma} \quad (3.11)$$

Where σ stands for variance and $S(t)$ represents an instant of the sample at time t and is similar to X_i in the previous equations. [16]

- Second difference

Second difference is defined as follows

$$\gamma = \frac{1}{T-2} \sum_{t=1}^{T-2} |S(t+2) - S(t)| \quad (3.12)$$

Here $S(t)$ represents an instant of the sample at time t and is similar to X_i in the previous equations. [16]

- Normalized Second difference

Normalized second difference can be defined by the following-

$$\bar{\gamma} = \frac{\frac{1}{T-2} \sum_{t=1}^{T-2} |S(t+2) - S(t)|}{\sigma} \quad (3.13)$$

Where σ stands for variance and $S(t)$ represents an instant of the sample at time t and is similar to X_i in the previous equations. [16]

- Shanon Entropy

This feature is popularly used as it quantifies the uncertainty of a signal. [35] It can be represented by the following equation as mentioned in [35].

$$E = - \sum_{t=1}^T (p_k \log[p_k]) \quad (3.14)$$

- Log Energy

It is the log of the energy term calculated as 3.15. It can be calculated as follows-

$$LogEnergy = \log\left(\sum_{i=1}^N (X_i)\right) \quad (3.15)$$

where X_i refers to the i th instant and N refers to the total number of instances in the data set. [33]

- Entropy

It is also a measure which is statistical in nature and can be considered a quantification of the randomness of a signal.

- Auto-correlation

It is essentially a correlation of a signal with itself or more precisely a delayed version of itself. In simple terms it is a measure of the similarity between the what is observed over time, as a function of time.

3.2.2 Spectral Features

The spectral features are as follows:

- Spectral Centroid

This is based on the statistical moments of the EEG signals and has been obtained using MATLAB function and is defined as -

$$\mu_1 = S_c = \frac{\sum_{k=b_1}^{b_2} f_k s_k}{\sum_{k=b_1}^{b_2} s_k} \quad (3.16)$$

Where f_k is frequency in Hz, s_k is the spectral value and b_1 and b_2 are the band edges over which the centroid is calculated. [36]

- Spectral Width

It is also known as spectral spread and is defined as follows -

$$\mu_2 = \sqrt{\frac{\sum_{k=b_1}^{b_2} (f_k - \mu_1)^2 s_k}{\sum_{k=b_1}^{b_2} s_k}} \quad (3.17)$$

Where all the variables have the same meaning as in Equation 3.16 [36]

- Spectral Asymmetry

It is also known as Spectral skewness and has the following equation -

$$\mu_3 = \sqrt{\frac{\sum_{k=b_1}^{b_2} (f_k - \mu_1)^3 s_k}{\mu_2^3 \sum_{k=b_1}^{b_2} s_k}} \quad (3.18)$$

Where all the variables have the same meaning as in Equation 3.17 [36]

- Spectral flatness

It is synonymous to spectral kurtosis and is defined as follows-

$$\mu_4 = \sqrt{\frac{\sum_{k=b1}^{b2} (f_k - \mu_1)^4 s_k}{\mu_2^4 \sum_{k=b1}^{b2} s_k}} \quad (3.19)$$

Where all the variables have the same meaning as in Equation 3.18. [36]

- Spectral Slope

$$slope = \frac{\sum_{k=b1}^{b2} (f_k - \mu_f)(s_k - \mu_s)}{\sum_{k=b1}^{b2} (f_k - \mu_f)} \quad (3.20)$$

Where f_k is frequency in Hz μ_f, μ_s represents mean frequency and mean spectral value while rest of the symbols have the same meaning as explained earlier. [36]

- Spectral Decrease

$$decrease = \frac{\sum_{k=b1}^{b2} \frac{(s_k - s_{b1})}{(k-1)}}{\sum_{k=b1+1}^{b2} (s_k)} \quad (3.21)$$

Where all the variables in the equation have the same meaning as explained above. [36]

3.2.3 Event Related Potential

The event related potential features are as follows:

- P100

It is often referred to as P1 also and is the first positive peak which is observed after a person has been exposed to a stimulus and usually occurs between 80 ms to 120 ms.

- N100

N100 is also observed between 80 ms to 120 ms but is the first negative peak or the first minimum encountered.

- P200

It is essentially the second positive peak which is observed between 150 ms to 275 ms or approximately around 200 ms and hence the name.

- N200

N200 is the negative peak of the minimum observed around 200 ms recorded between 150 ms to 275 ms

- P300

P300 is the third positive peak and is observed between 275 ms to 500 ms but usually falls around 300 ms.

The discrete wavelet features are as follows-Energy, Variance, Waveform Length and Entropy.

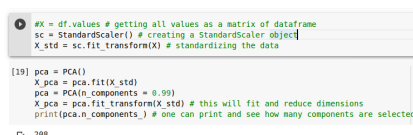
3.3 Feature Selection

There are numerous feature selection as well as feature reduction methods as observed during literature survey. The most commonly used being PCA, mRMR and random forest. In this project the following were used-

- PCA It is an unsupervised dimensionality reduction technique where only the principle components of the signals are retained and an uncorrelated set is obtained. While this method is commonly used, it helps to reduce the computational load. [3]

PCA converts the features into an alternate state space and uses principal components to represent the data. As less as 208 components can represent almost 99% of the data. The figure below shows the number to components extract to represent 99% of the data.

PCA works better when the data has been zero-meaned. It is also to be noted that



```

#X = df.values # getting all values as a matrix of dataframe
sc = StandardScaler() # creating a StandardScaler object
X_std = sc.fit_transform(X) # standardizing the data

[19]: pca = PCA()
      X_pca = pca.fit(X_std)
      pca = PCA(n_components = 0.99)
      X_pca = pca.fit_transform(X_std) # this will fit and reduce dimensions
      print(pca.n_components_) # one can print and see how many components are selected.
208

```

Figure 3.1: Number of principle components to represent 99% of the data

the test samples must also be zero-meaned and then the principle components have to be obtained to test the classifier model trained with the PCA components of the training data. The above picture also show how the data has been zero-meaned and variance has been set to 1. There are built in functions, both in Python as well as MATLAB which can be used for the principle component analysis which implicitly uses SVD.

- mRMR It is a supervised technique. This method uses mutual information and tries to minimize it while simultaneously optimizing maximal relevance which refers to each label having maximum mutual information with the features. In other words it ensures that

the features are representative of the labels but overlap minimally. [3]

mRMR was used for feature selection and used in conjunction with SVM for preliminary testing. While it was neither computationally too exhaustive nor time consuming, it did not provide a considerable improvement in accuracy when used in conjunction with the proposed classifiers. It only served the purpose of better understanding the feature space.

- **Random Forest** It is another supervised technique which takes into consideration the labels and provides a list of features with it's importance. It uses an ensemble of decision trees to perform the same and hence the name. Random forest can be used to find the importance of each feature. The following figure shows all the features and their individual importance plotted. It also acts as a classifier and has been explained in details in the next section which deals with classifiers.

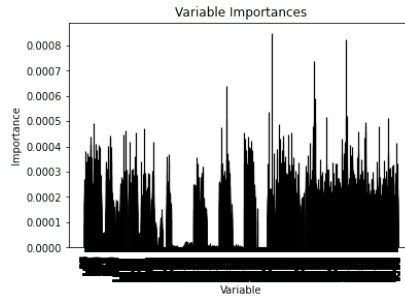


Figure 3.2: A plot of features against its importance

Brute force method can also be used for feature selection. It ensures the highest accuracy for a particular machine learning algorithm but is prone to over-fitting. More over the method is computationally exhaustive and not memory efficient and hence is not preferred especially when the number of features is large. This method was implemented but was not used after initial testing with decision trees as it proved to be computationally inefficient.

3.4 Classification

Various machine learning algorithms have been proposed and used on the data-set to obtain the best classifier. From the literature survey it was apparent that the most commonly used techniques included KNN, LDA, SVM, Bayes Net and Neural Networks on this particular data set. In general EEG signals have been classified most commonly with SVM, some ensemble techniques like random forest and others that use ensemble model and a growing trend in using deep learning techniques. Hence, in this project we have used SVM, KNN, Random

Forest, Adaboost and Neural Networks. It is also noteworthy that Bayes Networks failed in classification when used with the feature set extracted in this project.

3.4.1 SVM

Support vector machines have various alternatives that use different planes depending on the kernel and how well a kernel performs depends on the data set. Some commonly used kernels include polynomial kernels like quadratic kernels, radial and Gaussian kernels to name a few. SVM does not perform well with this data. Different SVM models were used as well as an ensemble of SVM classifiers were used together in a voting classifier with varying weights on each SVM classifier which has a different kernel. It does not give results which are comparable to those of random forest or adaboost. The following figure shows the performance of a single SVM model

```
[31] from sklearn import svm
      clf = svm.SVC()
      clf.fit(X, y)
      print("Training")
      print(clf.score(X, y))
      print("Testing")
      print(clf.score(X_test, y_test))
```

Training
0.3472222222222222
Testing
0.3548387096774194

Figure 3.3: Code, training and testing accuracy of a single SVM classifier

3.4.2 KNN

It takes into consideration the 'k' nearest neighbors to make the decision of classifying the particular sample input to a class. k is a hyper-parameter that can be set. Usually any number of nearest neighbors can be used. Some commonly used values range between three to ten.

In this particular project a bag of KNN classifiers were used for classification. This ensemble model performs much better than using just a single KNN model. The following images show the training and testing accuracy of a single KNN 3.4 while 3.5 shows that of ensemble KNN. As expected, it is more than the accuracy of a single KNN model.

```
[34] from sklearn.neighbors import KNeighborsClassifier
      clf = KNeighborsClassifier()
      clf.fit(X, y)
      print("Training")
      print(clf.score(X, y))
      print("Testing")
      print(clf.score(X_test, y_test))
```

Training
0.5944444444444444
Testing
0.45181798322580644

Figure 3.4: Training and testing accuracy of ensemble KNN

```

from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
bagging = BaggingClassifier(KNeighborsClassifier(),
                            max_samples=0.5, max_features=0.5, oob_score=True, bootstrap=True, bootstrap_features=True)
bagging.fit(X_train, y_train)
print("Training")
print(bagging.score(X_train, y_train))
print("Testing")
bagging.score(X_test, y_test)

```

Training Accuracy: 0.8721891782448244
Testing Accuracy: 0.3870867742323484

Figure 3.5: Training and testing accuracy of ensemble KNN

3.4.3 Random Forest

Random forest is an ensemble method that uses various decision trees to classify a sample. It has various hyper parameters like the number of estimators, the maximum depth of an estimator etc. It performs both feature selection as well as classification. It is composed of various estimators which are all decision trees, the visualization of a single decision tree would be as follows Since Random Forest has various parameters it can be tuned to obtain the best train-

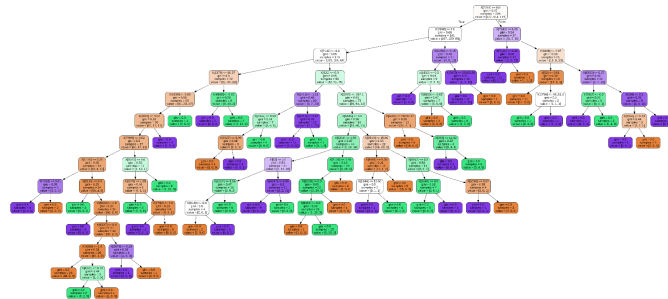


Figure 3.6: A single estimator from the Random Forest

ing/ testing accuracy. This can be done with the help of some search algorithms. First random search is used to find the range of parameters around which the best accuracy may be obtained. Then a grid search is performed in that region.

After performing random and grid search it has been observed that for the given data set, the best performance of the random forest classified is obtained when the number of estimators is 100, minimum number of samples split is set as 2, the minimum samples leaf is 3, maximum number of features used is square root of the total number of features while the maximum depth is 4614. Boot strap was used.

Since random search was used to decide the hyper parameters for the model, there is a probability that the model is stuck in a local minima and hence the training accuracy could not improve over 39.16% but the testing accuracy was 50.49% . It might be due to the size of the data set being small. There were other variations of random forest which had higher training accuracy but the testing accuracy was lesser. Following image shows a random forest classifier with none of the hyper parameters specified. From the training and testing accuracy, it can be

concluded that over fitting may be happening though generally, random forest is not prone to over fitting.

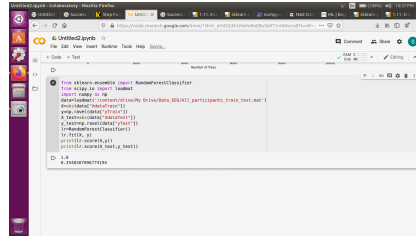


Figure 3.7: Random Forest

3.4.4 Ada-Boost

Ada Boost stands for adaptive boosting which involves a technique of using multiple weak algorithms to come up with a strong classifier. The number of classifiers can be as many as the number of features. This technique is very strong and forms one of the best ensemble models available. It was used without applying any feature reduction technique or extra regularization. The best test results were obtained with number of estimators equal to the number of features. Ada Boost performed better without PCA. PCA reduced the testing and training accuracy. The

```
[ ] clf = AdaBoostClassifier(n_estimators=9548, random_state=0)
    clf.fit(X, y)
    #AdaBoostClassifier(n_estimators=208, random_state=0)
    #print(clf.feature_importances_)
    print(clf.score(X, y))
    X_test=abs(data["XdataTest"])
    y_test=np.ravel(data["yTest"])
    clf.score(X_test, y_test)

0.9944444444444445
0.5161298322580645
```

Figure 3.8: Ada-boost architecture and performance

```
sc = StandardScaler() # creating a StandardScaler object
X_std = sc.fit_transform(X) # standardizing the data
pca = PCA()
X_pca = pca.fit(X_std)
pca = PCA(n_components = 0.99)
X_pca = pca.fit_transform(X_std) # this will fit and reduce dimensions
print(pca.n_components_) # one can print and see how many components are selected.
X_train, X_test, y_train, y_test = train_test_split(X_pca, y_test, size=0.1, random_state=0)

[43] clf = AdaBoostClassifier(n_estimators=208, random_state=0)
    clf.fit(X_train, y_train)
    #AdaBoostClassifier(n_estimators=208, random_state=0)
    #print(clf.feature_importances_)
    print(clf.score(X_pca, y))
    clf.score(X_test, y_test)

0.9855555555555556
0.2857142857142857
```

Figure 3.9: Training and testing accuracy of Adaboost with PCA

following figure shows how the test error reduces as the number of trees in the model increase.

3.4.5 Artificial Neural Networks

A neural network has been used which consists of 3 layers with number of neurons equal to the number of features in the first layer while the number of neurons in the second layers is half

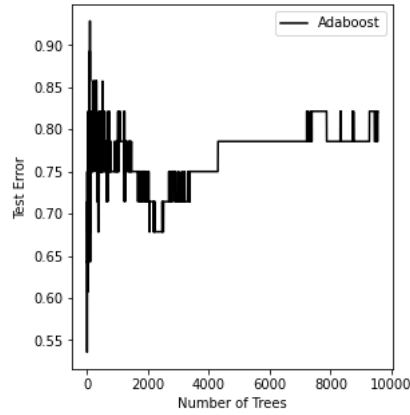


Figure 3.10: Test error as the number of trees in Adaboost increases

and the third layer is one-fourth. All the layers have batch normalization and a drop out of 0.2 to ensure there is no over-fitting. This is a rather shallow network and the input given to it is the extracted features and not the raw data. The architecture of the model can be summarized as in the figure below.

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 9548)	91173852
batch_normalization_1 (Batch Normalization)	(None, 9548)	38192
dropout_1 (Dropout)	(None, 9548)	0
dense_2 (Dense)	(None, 4774)	45586926
batch_normalization_2 (Batch Normalization)	(None, 4774)	19096
dropout_2 (Dropout)	(None, 4774)	0
dense_3 (Dense)	(None, 4774)	22795850
batch_normalization_3 (Batch Normalization)	(None, 4774)	19096
dropout_3 (Dropout)	(None, 4774)	0
dense_4 (Dense)	(None, 4)	19100
Total params: 159,652,112		
Trainable params: 159,613,920		
Non-trainable params: 38,192		

Figure 3.11: Architecture of the artificial neural network

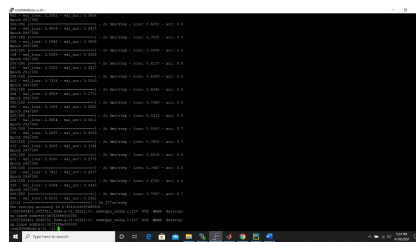


Figure 3.12: Accuracy obtained

Chapter 4

Results

The results obtained using the proposed methodology to predict the emotion of the participants is summarized in the column below. The models are not subject dependent and is a general model applicable to all participants and all sessions. For testing, a held out data set which was randomly chosen for testing. There are three classes of emotions being classified.

Table 4.1: Comparison of accuracy of the best model using each algorithm

Comparison of results		
Machine Learning Algorithm	Training Accuracy	Testing Accuracy
SVM	34.72%	35.48
KNN ensemble	67.21%	38.70%
Random Forest	39.16%	50.49%
Adaboost	99.4%	52%
Artificial Neural Network	74%	48%

It has been observed that while PCA reduces the time taken for computation as it reduces the dimensionality of the data. it does not provide a big improvement in the accuracy obtained.

The ensemble models perform better than other models which is as expected, further it can be observed that while the training accuracy vary largely between the different models, the test accuracy does not vary as much. This leads to the conclusion that some models are over-fitting.

Use more cross-validation has improved the accuracy of some models while others are not so affected by it. A cross-validation of 3, 5, 10 and even 20 has been experimented with.

While the accuracy of the models are better than chance predictions, showing that the pre-processing and feature extraction methods are adequate, the not so impressive accuracies might

be a result of using a small data set with just 390 samples. While the samples are very representative of the EEG signals the size of the data set is a concern. Better results can be obtained with larger data sets.

While the Artificial neural network used is a shallow network, it shows promising results, very close to what is obtained using Adaboost and hence is very promising.

When compared to the accuracy shown in the reference papers for this data set for all participants and all sessions, [31] obtained 51% accuracy and [37] obtained 40% the accuracy of the Adaboost model is better.

Chapter 5

Conclusion

From the results obtained, it can be observed that EEG signals can provide vital information about the emotional state of a human being. There is scope for better results with more vigorous data-collection and use of deeper neural networks which are better at learning the non-linearity of the systems. Further, by under taking data collection, there can be a better understanding of the noise present in the data hence making pre-processing more efficient. With more efficient pre-processing there will be cleaner data which can provide better results. Further just the increase in size of the data set can make a difference. Lastly, the most intriguing direction for emotion detection would be the use of deep learning specifically using convolutional layers and recurrent layers on the raw data set whereby completely by-passing the idea of using hand crafted feature selection techniques altogether. This project work is merely a first step towards the creation of a resilient machine learning algorithm which will enable the existence of emotionally intelligent systems.

Bibliography

- [1] S Jerritta, M Murugappan, R Nagarajan, and Khairunizam Wan. Physiological signals based human emotion recognition: a review. In *2011 IEEE 7th International Colloquium on Signal Processing and its Applications*, pages 410–415. IEEE, 2011.
- [2] Munaza Ramzan and Suma Dawn. A survey of brainwaves using electroencephalography (eeg) to develop robust brain-computer interfaces (bcis): Processing techniques and algorithms. In *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, pages 642–647. IEEE, 2019.
- [3] Jingxin Liu, Hongying Meng, Maozhen Li, Fan Zhang, Rui Qin, and Asoke K Nandi. Emotion detection from eeg recordings based on supervised and unsupervised dimension reduction. *Concurrency and Computation: Practice and Experience*, 30(23):e4446, 2018.
- [4] Amirhossein H Memar and Ehsan T Esfahani. Physiological measures for human performance analysis in human-robot teamwork: Case of tele-exploration. *IEEE Access*, 6:3694–3705, 2018.
- [5] Stefan Ehrlich, Agnieszka Wykowska, Karinne Ramirez-Amaro, and Gordon Cheng. When to engage in interaction—and how? eeg-based enhancement of robot’s ability to sense social signals in hri. In *2014 IEEE-RAS International Conference on Humanoid Robots*, pages 1104–1109. IEEE, 2014.
- [6] Derek McColl, Alexander Hong, Naoaki Hatakeyama, Goldie Nejat, and Beno Benhabib. A survey of autonomous human affect detection methods for social robots engaged in natural hri. *Journal of Intelligent & Robotic Systems*, 82(1):101–133, 2016.
- [7] Kerstin Dautenhahn. Methodology & themes of human-robot interaction: A growing research field. *International Journal of Advanced Robotic Systems*, 4(1):15, 2007.

- [8] James E Young, JaYoung Sung, Amy Volda, Ehud Sharlin, Takeo Igarashi, Henrik I Christensen, and Rebecca E Grinter. Evaluating human-robot interaction. *International Journal of Social Robotics*, 3(1):53–67, 2011.
- [9] Abeer Al-Nafjan, Manar Hosny, Yousef Al-Ohali, and Areej Al-Wabil. Review and classification of emotion recognition based on eeg brain-computer interface system research: a systematic review. *Applied Sciences*, 7(12):1239, 2017.
- [10] Lars Rune Christensen and Mohamed Ahmed Abdullah. Eeg emotion detection review. In *2018 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*, pages 1–7. IEEE, 2018.
- [11] Soraia M Alarcao and Manuel J Fonseca. Emotions recognition using eeg signals: A survey. *IEEE Transactions on Affective Computing*, 2017.
- [12] Anita Patil, Chinmayee Deshmukh, and AR Panat. Feature extraction of eeg for emotion recognition using hjorth features and higher order crossings. In *2016 Conference on Advances in Signal Processing (CASP)*, pages 429–434. IEEE, 2016.
- [13] Murugappan Murugappan, Nagarajan Ramachandran, and Yaacob Sazali. Classification of human emotion from eeg using discrete wavelet transform. *Journal of biomedical science and engineering*, 3(04):390, 2010.
- [14] Murugappan Murugappan, Ramachandran Nagarajan, and Sazali Yaacob. Combining spatial filtering and wavelet transform for classifying human emotions using eeg signals. *Journal of Medical and Biological Engineering*, 31(1):45–51, 2011.
- [15] Abeer Al-Nafjan, Manar Hosny, Areej Al-Wabil, and Yousef Al-Ohali. Classification of human emotions from electroencephalogram (eeg) signal using deep neural network. *International Journal of Advanced Computer Science and Applications*, 8(9):419–425, 2017.
- [16] Jingxin Liu, Hongying Meng, Asoke Nandi, and Maozhen Li. Emotion detection from eeg recordings. In *2016 12th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*, pages 1722–1727. IEEE, 2016.
- [17] Adrian Qi-Xiang Ang, Yi Qi Yeong, and Wee Wee. Emotion classification from eeg signals using time-frequency-dwt features and ann. *J Comput Commun*, 5(03):75, 2017.

- [18] Pascal Ackermann, Christian Kohlschein, Jó Ágila Bitsch, Klaus Wehrle, and Sabina Jeschke. Eeg-based automatic emotion recognition: Feature extraction, selection and classification methods. In *2016 IEEE 18th international conference on e-health networking, applications and services (Healthcom)*, pages 1–6. IEEE, 2016.
- [19] Yang Wei, Yue Wu, and John Tudor. A real-time wearable emotion detection headband based on eeg measurement. *Sensors and Actuators A: Physical*, 263:614–621, 2017.
- [20] Debashis Das Chakladar and Sanjay Chakraborty. Eeg based emotion classification using “correlation based subset selection”. *Biologically inspired cognitive architectures*, 24:98–106, 2018.
- [21] SG Mangalagowri and P Cyril Prasanna Raj. Eeg feature extraction and classification using feed forward backpropagation algorithm for emotion detection. In *2016 International Conference on Electrical, Electronics, Communication, Computer and Optimization Techniques (ICEECOT)*, pages 183–187. IEEE, 2016.
- [22] Ning Zhuang, Ying Zeng, Li Tong, Chi Zhang, Hanming Zhang, and Bin Yan. Emotion recognition from eeg signals using multidimensional information in emd domain. *BioMed research international*, 2017, 2017.
- [23] Yannick Roy, Hubert Banville, Isabela Albuquerque, Alexandre Gramfort, Tiago H Falk, and Jocelyn Faubert. Deep learning-based electroencephalography analysis: a systematic review. *Journal of neural engineering*, 2019.
- [24] Swati Vaid, Preeti Singh, and Chamandeep Kaur. Classification of human emotions using multiwavelet transform based features and random forest technique. *Indian Journal of Science and Technology*, 8(28):1–7, 2015.
- [25] Memoona Iftikhar, Shoab Ahmad Khan, and Ali Hassan. A survey of deep learning and traditional approaches for eeg signal processing and classification. In *2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 395–400. IEEE, 2018.
- [26] Zirui Lan, Olga Sourina, Lipo Wang, Yisi Liu, Reinhold Scherer, and Gernot R Müller-Putz. Stable feature selection for eeg-based emotion recognition. In *2018 International Conference on Cyberworlds (CW)*, pages 176–183. IEEE, 2018.

- [27] Arman Savran, Koray Ciftci, Guillaume Chanel, Javier Mota, Luong Hong Viet, Blent Sankur, Lale Akarun, Alice Caplier, and Michele Rombaut. Emotion detection in the loop from brain signals and facial images. In *Proceedings of the eNTERFACE 2006 Workshop*, 2006.
- [28] Anne-Claire Conneau and Slim Essid. Assessment of new spectral features for eeg-based emotion recognition. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4698–4702. IEEE, 2014.
- [29] Mandeep Singh, Mr Mooninder Singh, and Nikhil Singhal. Emotion recognition along valence axis using naive bayes classifier. *International Journal of Information Technology & Knowledge Management*, 7(1):51–55, 2013.
- [30] Mandeep Singh, Mr Mooninder Singh, and Nikhil Singhal. Emotion recognition along valence axis using naive bayes classifier. *International Journal of Information Technology & Knowledge Management*, 7(1):51–55, 2013.
- [31] Z Khalili and MH Moradi. Emotion detection using brain and peripheral signals. In *2008 Cairo international biomedical engineering conference*, pages 1–4. IEEE, 2008.
- [32] Mandeep Singh, Mr Mooninder Singh, and Nikhil Singhal. Comparison of classifiers with event related potentials as an attribute in emotion quantification along valence axis using eeg signals. *International Journal of Information Technology & Knowledge Management*, 7(1):61–74, 2013.
- [33] Hazirah Hasni, Norashikin Yahya, Vijanth S Asirvadam, and Munsif Ali Jatoi. Analysis of electromyogram (emg) for detection of neuromuscular disorders. In *2018 International Conference on Intelligent and Advanced System (ICIAS)*, pages 1–6. IEEE, 2018.
- [34] Angkoon Phinyomark, Pornchai Phukpattaranont, and Chusak Limsakul. Feature reduction and selection for emg signal classification. *Expert systems with applications*, 39(8):7420–7431, 2012.
- [35] Swati Vaid, Preeti Singh, and Chamandeep Kaur. Classification of human emotions using multiwavelet transform based features and random forest technique. *Indian Journal of Science and Technology*, 8(28):1–7, 2015.

-
- [36] Geoffroy Peeters. A large set of audio features for sound description (similarity and classification) in the cuidado project. *CUIDADO IST Project Report*, 54(0):1–25, 2004.
- [37] Yun Su, Bin Hu, Lixin Xu, Hanshu Cai, Philip Moore, Xiaowei Zhang, and Jing Chen. Emotiono+: Physiological signals knowledge representation and emotion reasoning model for mental health monitoring. pages 529–535, 2014.

Appendix A

MATLAB Code

A.1 Preprocessing and Feature Extraction main

A.1.1 Session 1

```
for main_loop_counter=1:1:3
clearvars -except main_loop_counter

%% Extracting EEG Signals
%Participant-1/session-1 %NOT CONSIDERED, DIFFERENT SAMPLING FREQUENCY
%[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim]
%= eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\','all','n')
% % Participant1, session 1
% m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part1_IAPS_SES1_EEG_fNIRS_030820_
↳ 06.bdf.mrk.csv')
switch main_loop_counter

%Participant-2/session-1
%[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim1] =
↳ eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part2_IAPS_SES1_EEG_fNIRS_070_
↳ 82006.bdf','all','n')
% %Participant-2/session-1
```

```

%m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part2_IAPS_SES1_EEG_fNIRS_0708200」
↳ 6.bdf.mrk.csv')

    case 1

%Participant-3/session-1

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim1] =
↳ eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part3_IAPS_SES1_EEG_fNIRS_070」
↳ 82006.bdf','all','n')

%Participant-3/session-1

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part3_IAPS_SES1_EEG_fNIRS_07082006」
↳ .bdf.mrk.csv')

    case 2

%Participant-4/session-1

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('eINTERF」
↳ ACE06_EMOBRAIN\Data\EEG\Part4_IAPS_SES1_EEG_fNIRS_07082006.bdf','all','n')

% %Participant-4/session-1

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part4_IAPS_SES1_EEG_fNIRS_07082006」
↳ .bdf.mrk.csv')

    case 3

%Participan-5/session-1

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('eINTERF」
↳ ACE06_EMOBRAIN\Data\EEG\Part5_IAPS_SES1_EEG_fNIRS_08082006.bdf','all','n')

% %Participant-5/session-1

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part5_IAPS_SES1_EEG_fNIRS_08082006」
↳ .bdf.mrk.csv')

end

%% reading class labels

labels_class

↳ =readtable('eINTERFACE06_EMOBRAIN\Data\Common\IAPS_Classes_EEG_fNIRS.csv')

%% PRE-PROCESSING

data=data'

```

```

%% taking chunks a different way
% Using the markers to assign labels by considering the label 1 as start of
% video and the following label as start of logic
no_of_rows_data= size(data,1) %No of total samples
no_of_columns_data= size(data,2)%No of electrodes

no_of_rows_m=size(m,1)%No of classes
no_of_columns_m=size(m,2)% No of markers related to each class

j=1
i=1
k=1
data_chunk_fromsynch={}
while i<= no_of_rows_data
    l= m(j,1)-1
    if i~=1
        data_chunk_fromsynch{k}= data(i:l,1:64)
        k=k+1
    end
    i=m(j,1)
    j=j+2
    if j>no_of_rows_m
        break
    end
end

length(data_chunk_fromsynch)

%% Annoting the data with labels
cl=zeros(30,1)
for i=1:1:30
    label_class=labels_class{i,1}
    if strcmp(label_class,'Calm')==1

```

```

        cl(i)=1; %CALM IS CLASS 1
    elseif strcmp(label_class,'Pos')==1
        cl(i)=2; %POSITIVE IS CLASS 2
    elseif strcmp(label_class,'Neg') ==1
        cl(i)=3;%NEGATIVE IS CLASS 3
    end
end

%% feature extraction
%Zero meaning
zero_meaned_datachunk=zeromeaning(data_chunk_fromsynch)
clear data_chunk_fromsynch
%% Fourier transform
frequency_d_data = fourier_transform(zero_meaned_datachunk)
clear zero_meaned_datachunk
%% trying butterworth
% Wn=0.8
% [b,a] = butter(3, Wn, 'low');
% filteredSignal = filter(b, a, zero_meaned_data_f );
%filteredSignal = filteredSignal - mean(filteredSignal); % Subtracting the
↪ mean to block DC Component
%% Filter
%Keeping only alpha, beta, gamma,theta
% fcut = 60; %Hz
filtered_f_data = filtering_eeg(frequency_d_data,fs)
clear frequency_d_data
%% Getting time domain back
filtered_t_data=inverse_fourier(filtered_f_data)
clear frequency_f_data
% %% zero meaning i.e remove DC
% zero_meaned_data = zeromeaning(filtered_t_data)
%% plotting zero meaned data
% figure

```

```

% hold on
% %plot(data_chunk{1}(:,1),'--')
% plot(zero_meaned_data{1}(:,5))
% hold off

%% Extracting statistical features time domain
% [mean_t, std_t] = statistical_feature_extraction(zero_meaned_data)

%% zeromeaning frequency data
%zero_meaned_data_f = zeromeaning(filtered_f_data)

%% Taking absolute value
%abs_value=absolute_value(zero_meaned_data_f)

%% Extracting frequency domain
%[mean_f, std_f] = statistical_feature_extraction(abs_value)

%% Consolidated to use in the ml app
% z=[mean_f  std_f mean_t std_t cl]
% z_t=[mean_t  std_t  cl]
% z_f=[mean_f  std_f  cl]


% %% PSD
% Frequencies=1024
% Fs=1024
% %Hpsd = dspdata.psd(Data,Frequencies)
% for i=1:1:length(filtered_f_data)
%     xdft=filtered_f_data{i}
%     N = length(xdft);
%     %xdft = fft(x);
%     % xdft = xdft(1:(N/2+1),:);
%     psdx1 = (1/(Fs*N)) * abs(xdft).^2;
%     psdx=sum(psdx1,1)
%     % Data =data_chunk_fromsynch{i}
%     % Frequencies=length(Data)
%     % Hpsd = dspdata.psd(abs(Data),Frequencies)
%     psd{i}=psdx

```

```

% %      psd{i}=Hpsd
% end

% %psdx(2:end-1) = 2*psdx(2:end-1);
%% Another way to calculate PSD
Fsx =1024
dx=1/Fsx

for i=1:1:length(filtered_t_data)
    x=filtered_t_data{i}
    L = length(x);
    Lfft = 2^nextpow2(L);
    dfx = 1/(Lfft*dx);
    fourier_image = fftshift(fft(x,Lfft)/Lfft);
    PSDfromFFT = abs(fourier_image).^2;
    freq = -Fsx/2:dfx:Fsx/2-dfx; % Frequency axis
    freqs{i}=freq
    PSD{i}=PSDfromFFT
end

%% Extracting features from power spectral density (frequency domain )
% [mean_p, std_p] = statistical_feature_extraction(abs_value)
% [max_p,min_p] =max_min(PSD)

% %% whole set of statistical features
% features1 = extract(absolute_value(filtered_t_data))
% features2 = extract(absolute_value(filtered_f_data))
% features3 = extract(absolute_value(PSD))
% % Y

```

```

% trial=[features1 features2 features3 c1]
%% dwt
%   winsize:      window size (length of x)
%   wininc:       spacing of the windows (winsize)

for i=1:1:length(filtered_t_data)
    features_dw=[]
    for j=1:1:size(filtered_f_data{1},2)
        h= filtered_f_data{i}(:,j)
        winsize=length(h)
        SF=1024
        wininc=32
        feat = getmswtfeat(h,winsize,wininc,SF)
        features_dw=[features_dw feat]
    end
    features_discrete_wavelet(i,:)=features_dw
end

% %% Discrete Wavelet Transform
% wname='db1' %Haar
% for i=1:1:length(filtered_f_data)
%     cA=[];
%     cH=[];
%     cV=[];
%     cD=[];
%
%     for j=1:1:size(filtered_f_data{1},2)
%         h= filtered_f_data{i}(:,j)
%         [ca,ch,cv,cd] = dwt2(h,wname)
%         cA = [cA ca]
%         cH=[cH ch]

```

```

%      cV=[cV cv]
%      cD=[cD cd]
%      end
%      CA{i}=[cA]
%      CH{i}=[cH]
%      CV{i}=[cV]
%      CD{i}=[cD]
% %      ca(i) = cA
% %      ch(i)=cH
% %      cv(i)= cV
% %      cd(i)= cD
% end

%% whole set of statistical features
% features1 = extract(absolute_value(filtered_t_data))
% features2 = extract(absolute_value(filtered_f_data))
% features3 = extract(absolute_value(PSD))
% % %features_discrete_wavelet
% % %features4 = extract(absolute_value(PSD))
% % features=[features1 features2 features3 features_discrete_wavelet cl]
% features4=More_extract(filtered_t_data)
% features5=More_extract(filtered_f_data)
% features6=More_extract(PSD)
% %% New features
% features7=Spectral_Features_Extract(filtered_t_data,SF)
% features8=Spectral_Features_Extract(filtered_f_data,SF)
% features9=Spectral_Features_Extract(PSD,SF)
features1 = extract(absolute_value(filtered_t_data))
features2 = extract(absolute_value(fourier_transform(filtered_t_data)))
features3 = extract(absolute_value(PSD))
features4 = More_extract(filtered_t_data)
features5 = More_extract(fourier_transform(filtered_t_data))
features6 = More_extract(PSD)

```

```

features7=Spectral_Features_Extract(filtered_t_data,SF)
features8=Spectral_Features_Extract(fourier_transform(filtered_t_data),SF)
features9=Spectral_Features_Extract(PSD,SF)
features=[features1 features2 features3 features4 features5 features6
↳ features7 features8 features9 features_discrete_wavelet c1]
switch main_loop_counter

%PARTICIPANT 1 SESSION1 IS NOT CONSIDERED

%save ('participant1_session1','features1','features2','features3','features_」
↳ discrete_wavelet','features')

%save ('participant2_session1','features1','features2','features3','features_」
↳ discrete_wavelet','features')
    case 1
% save ('participant3_session1','features1','features2','features3','features_」
↳ _discrete_wavelet','features')
% save ('participant3_session1_more','features1','features2','features3','fea」
↳ tures')
save ('participant3_session1_4_13','features1','features2','features3','featu」
↳ res4','features5','features6','features7','features8','features9','featur」
↳ es_discrete_wavelet','features')
    case 2
% save ('participant4_session1','features1','features2','features3','features_」
↳ _discrete_wavelet','features')
% save ('participant4_session1_more','features1','features2','features3','fea」
↳ tures')
save ('participant4_session1_4_13','features1','features2','features3','featu」
↳ res4','features5','features6','features7','features8','features9','featur」
↳ es_discrete_wavelet','features')
    case 3
% save ('participant5_session1','features1','features2','features3','features_」
↳ _discrete_wavelet','features')

```

```
% save ('participant5_session1_more','features1','features2','features3','fea_
↳ tures')

save ('participant5_session1_4_13','features1','features2','features3','featu_
↳ res4','features5','features6','features7','features8','features9','featur_
↳ es_discrete_wavelet','features')

end

end
```

A.1.2 Session 2

```
for main_loop_counter=1:1:5

clearvars -except main_loop_counter

%% Extracting EEG Signals

switch main_loop_counter

    case 1

%Participant-1/session-2

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] =
↳ eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part1_IAPS_SES2_EEG_fNIRS_070_
↳ 82006.bdf','all','n');

save ('participant1_session2_RAW','data','numChan','labels','txt','fs','gain'
↳ , 'prefiltering','ChanDim')

    case 2

%Participant-2/session-2

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] =
↳ eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part2_IAPS_SES2_EEG_fNIRS_080_
↳ 82006.bdf','all','n');

save ('participant2_session2_RAW','data','numChan','labels','txt','fs','gain'
↳ , 'prefiltering','ChanDim')

    case 3

%Participant-3/session-2

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] =
↳ eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part3_IAPS_SES2_EEG_fNIRS_080_
↳ 82006.bdf','all','n');
```

```

save ('participant3_session2_RAW','data','numChan','labels','txt','fs','gain'
↪  , 'prefiltering','ChanDim')

    case 4

%Participant-4/session-2

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] =
↪  eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part4_IAPS_SES2_EEG_fNIRS_080'
↪  82006.bdf','all','n');

save ('participant4_session2_RAW','data','numChan','labels','txt','fs','gain'
↪  , 'prefiltering','ChanDim')

    case 5

%Participant-5/session-2

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] =
↪  eeg_read_bdf('eINTERFACE06_EMOBRAIN\Data\EEG\Part5_IAPS_SES2_EEG_fNIRS_090'
↪  82006.bdf','all','n');

save ('participant5_session2_RAW','data','numChan','labels','txt','fs','gain'
↪  , 'prefiltering','ChanDim')

end

%% Reading markers

switch main_loop_counter

    case 1

% Participant1, session 2

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part1_IAPS_SES2_EEG_fNIRS_07082006'
↪  .bdf.mrk.csv');

    case 2

%Participant-2/session-2

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part2_IAPS_SES2_EEG_fNIRS_08082006'
↪  .bdf.mrk.csv');

    case 3

%Participant-3/session-2

```

```

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part3_IAPS_SES2_EEG_fNIRS_08082006」
↳ .bdf.mrk.csv');

    case 4

%Participant-4/session-2

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part4_IAPS_SES2_EEG_fNIRS_08082006」
↳ .bdf.mrk.csv');

    case 5

%Participant-5/session-2

m = csvread('eINTERFACE06_EMOBRAIN\Data\EEG\Part5_IAPS_SES2_EEG_fNIRS_09082006」
↳ .bdf.mrk.csv');

end

%% reading class labels

labels_class

↳ =readtable('eINTERFACE06_EMOBRAIN\Data\Common\IAPS_Classes_EEG_fNIRS.csv');

%% PRE-PROCESSING

data=data';

%% taking chunks a different way

% Using the markers to assign labels by considering the label 1 as start of
% video and the following label as start of logic

no_of_rows_data= size(data,1); %No of total samples
no_of_columns_data= size(data,2);%No of electrodes

no_of_rows_m=size(m,1);%No of classes
no_of_columns_m=size(m,2);% No of markers related to each class

j=1;

i=1;

k=1;

data_chunk_fromsynch={};

while i<= no_of_rows_data

```

```

    l= m(j,1)-1;
    if i~=1
        data_chunk_fromsynch{k}= data(i:1,1:64);
        k=k+1;
    end
    i=m(j,1);
    j=j+2;
    if j>no_of_rows_m
        break
    end
end

clear data

%% Annoting the data with labels
cl=zeros(30,1);
for i=1:1:30
    label_class=labels_class{i,2}
    if strcmp(label_class,'Calm')==1
        cl(i)=1; %CALM IS CLASS 1
    elseif strcmp(label_class,'Pos')==1
        cl(i)=2; %POSITIVE IS CLASS 2
    elseif strcmp(label_class,'Neg')==1
        cl(i)=3;%NEGATIVE IS CLASS 3
    end
end

%% feature extraction
%Zero meaning
zero_meaned_datachunk=zeromeaning(data_chunk_fromsynch);
clear data_chunk_fromsynch

%% Fourier transform
frequency_d_data = fourier_transform(zero_meaned_datachunk);

```

```

clear zero_meaned_datachunk
%% trying butterworth
% Wn=0.8
% [b,a] = butter(3, Wn, 'low');
% filteredSignal = filter(b, a, zero_meaned_data_f );
%filteredSignal = filteredSignal - mean(filteredSignal); % Subtracting the
↪ mean to block DC Component
%% Filter
%Keeping only alpha, beta, gamma,theta
% fcut = 60; %Hz
filtered_f_data = filtering_eeg(frequency_d_data,fs);
clear frequency_d_data
%% Getting time domain back
filtered_t_data=inverse_fourier(filtered_f_data);
clear frequency_f_data
%% plotting zero meaned data
% figure
% hold on
% %plot(data_chunk{1}(:,1),'--')
% plot(zero_meaned_data{1}(:,5))
% hold off
%% Extracting statistical features time domain
% [mean_t, std_t] = statistical_feature_extraction(zero_meaned_data)
%% zeromeaning frequency data
% zero_meaned_data_f = zeromeaning(filtered_f_data)
%Dont think its necessary
%% Taking absolute value
%abs_value=absolute_value(filtered_f_data)
%% Extracting frequency domain
%[mean_f, std_f] = statistical_feature_extraction(abs_value)
%% Consolidated to use in the ml app
% z=[mean_f std_f mean_t std_t cl]

```

```

% z_t=[mean_t  std_t  cl]
% z_f=[mean_f  std_f  cl]

% %% PSD
% Frequencies=1024
% Fs=1024
% %Hpsd = dspdata.psd(Data,Frequencies)
% for i=1:1:length(filtered_f_data)
%     xdft=filtered_f_data{i}
%     N = length(xdft);
%     %xdft = fft(x);
%     % xdft = xdft(1:(N/2+1),:);
%     psdx1 = (1/(Fs*N)) * abs(xdft).^2;
%     psdx=sum(psdx1,1)
%     Data =data_chunk_fromsynch{i}
%     Frequencies=length(Data)
%     Hpsd = dspdata.psd(abs(Data),Frequencies)
%     psd{i}=psdx
%     % psd{i}=Hpsd
% end
% %psdx(2:end-1) = 2*psdx(2:end-1);
%% Another way to calculate PSD
Fsx =1024;
dx=1/Fsx;

for i=1:1:length(filtered_t_data)
    x=filtered_t_data{i}
    L = length(x);
    Lfft = 2^nextpow2(L);
    dfx = 1/(Lfft*dx);

```

```

    fourier_image = fftshift(fft(x,Lfft)/Lfft);
    PSDfromFFT    = abs(fourier_image).^2;
    freq          = -Fsx/2:dfx:Fsx/2-dfx; % Frequency axis
    freqs{i}=freq;
    PSD{i}=PSDfromFFT;

end

%% Extracting features from power spectral density (frequency domain )
% [mean_p, std_p] = statistical_feature_extraction(abs_value)
% [max_p,min_p] =max_min(PSD)

%% %% whole set of statistical features
% features1 = extract(absolute_value(filtered_t_data))
% features2 = extract(absolute_value(filtered_f_data))
% features3 = extract(absolute_value(PSD))
%% Y
% trial=[features1 features2 features3 cl]
%% dwt
%   winsize:      window size (length of x)
%   wininc:       spacing of the windows (winsize)

for i=1:1:length(filtered_f_data)
    features_dw=[];
    for j=1:1:size(filtered_f_data{1},2)
        h= filtered_f_data{i}(:,j);
        winsize=length(h);
        SF=1024;
        wininc=32;
        feat = getmswtfeat(h,winsize,wininc,SF);
        features_dw=[features_dw feat];
    end
end

```

```

        end
features_discrete_wavelet(i,:)=features_dw;
end

% %% Discrete Wavelet Transform
% wname='db1' %Haar
% for i=1:1:length(filtered_f_data)
%
%         cA=[];
%         cH=[];
%         cV=[];
%         cD=[];
%
%         for j=1:1:size(filtered_f_data{1},2)
%             h= filtered_f_data{i}(:,j)
%             [ca,ch,cv,cd] = dwt2(h,wname)
%             cA = [cA ca]
%             cH=[cH ch]
%             cV=[cV cv]
%             cD=[cD cd]
%         end
%         CA{i}=[cA]
%         CH{i}=[cH]
%         CV{i}=[cV]
%         CD{i}=[cD]
%
%         ca(i) = cA
%         ch(i)=cH
%         cv(i)= cV
%         cd(i)= cD
%     end
%% whole set of statistical features
% features1 = extract(absolute_value(filtered_t_data))

```

```

% features2 = extract(absolute_value(filtered_f_data))
% features3 = extract(absolute_value(PSD))
% features4=More_extract(filtered_t_data)
% features5=More_extract(filtered_f_data)
% features6=More_extract(PSD)

features1 = extract(absolute_value(filtered_t_data))
features2 = extract(absolute_value(fourier_transform(filtered_t_data)))
features3 = extract(absolute_value(PSD))
features4 = More_extract(filtered_t_data)
features5 = More_extract(fourier_transform(filtered_t_data))
features6 = More_extract(PSD)
features7=Spectral_Features_Extract(filtered_t_data,SF)
features8=Spectral_Features_Extract(fourier_transform(filtered_t_data),SF)
features9=Spectral_Features_Extract(PSD,SF)
%features_discrete_wavelet
%features4 = extract(absolute_value(PSD))
% features=[features1 features2 features3 features_discrete_wavelet c1]
features=[features1 features2 features3 features4 features5 features6
↪ features7 features8 features9 features_discrete_wavelet c1]
switch main_loop_counter
    case 1
% save ('participant1_session2','features1','features2','features3','features_
↪ _discrete_wavelet','features')
% save ('participant1_session2_more','features1','features2','features3','fea
↪ tures')
save ('participant1_session2_4_13','features1','features2','features3','featu
↪ res4','features5','features6','features7','features8','features9','featur
↪ es_discrete_wavelet','features')
    case 2
% save ('participant2_session2','features1','features2','features3','features_
↪ _discrete_wavelet','features')

```

```

% save ('participant2_session2_more','features1','features2','features3','fea_
↳ tures')

save ('participant2_session2_4_13','features1','features2','features3','featu_
↳ res4','features5','features6','features7','features8','features9','featur_
↳ es_discrete_wavelet','features')

    case 3

% save ('participant3_session2','features1','features2','features3','features_
↳ _discrete_wavelet','features')

% save ('participant3_session2_more','features1','features2','features3','fea_
↳ tures')

save ('participant3_session2_4_13','features1','features2','features3','featu_
↳ res4','features5','features6','features7','features8','features9','featur_
↳ es_discrete_wavelet','features')

    case 4

% save ('participant4_session2','features1','features2','features3','features_
↳ _discrete_wavelet','features')

% save ('participant4_session2_more','features1','features2','features3','fea_
↳ tures')

save ('participant4_session2_4_13','features1','features2','features3','featu_
↳ res4','features5','features6','features7','features8','features9','featur_
↳ es_discrete_wavelet','features')

    case 5

% save ('participant5_session2','features1','features2','features3','features_
↳ _discrete_wavelet','features')

% save ('participant5_session2_more','features1','features2','features3','fea_
↳ tures')

save ('participant5_session2_4_13','features1','features2','features3','featu_
↳ res4','features5','features6','features7','features8','features9','featur_
↳ es_discrete_wavelet','features')

end

main_loop_counter

end

```

A.1.3 Session 3

```

for main_loop_counter=1:1:5
clearvars -except main_loop_counter
%% Extracting EEG Signals
switch main_loop_counter
    case 1
%% Extracting EEG Signals
%Participant-1/session-3
[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('ENTERF」
↪ ACE06_EMOBRAIN\Data\EEG\Part1_IAPS_SES3_EEG_fNIRS_08082006.bdf','all','n')
save ('participant1_session3_RAW','data','numChan','labels','txt','fs','gain」
↪ , 'prefiltering','ChanDim')
    case 2
% %Participant-2/session-3
[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('ENTERF」
↪ ACE06_EMOBRAIN\Data\EEG\Part2_IAPS_SES3_EEG_fNIRS_09082006.bdf','all','n')
save ('participant2_session3_RAW','data','numChan','labels','txt','fs','gain」
↪ , 'prefiltering','ChanDim')
    case 3
%Participant-3/session-3
[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('ENTERF」
↪ ACE06_EMOBRAIN\Data\EEG\Part3_IAPS_SES3_EEG_fNIRS_08082006.bdf','all','n')
save ('participant3_session3_RAW','data','numChan','labels','txt','fs','gain」
↪ , 'prefiltering','ChanDim')
    case 4
%Participant-4/session-3
[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('ENTERF」
↪ ACE06_EMOBRAIN\Data\EEG\Part4_IAPS_SES3_EEG_fNIRS_09082006.bdf','all','n')
save ('participant4_session3_RAW','data','numChan','labels','txt','fs','gain」
↪ , 'prefiltering','ChanDim')
    case 5

```

```

%Participant-5/session-3

[data,numChan,labels,txt,fs,gain,prefiltering,ChanDim] = eeg_read_bdf('eNTERF」
↳ ACE06_EMOBRAIN\Data\EEG\Part5_IAPS_SES3_EEG_fNIRS_09082006.bdf','all','n')
save ('participant5_session3_RAW','data','numChan','labels','txt','fs','gain」
↳ , 'prefiltering','ChanDim')
end

%% Reading markers
switch main_loop_counter
    case 1
% Participant1, session 3
m = csvread('eNTERFACE06_EMOBRAIN\Data\EEG\Part1_IAPS_SES3_EEG_fNIRS_08082006」
↳ .bdf.mrk.csv')
    case 2
%Participant-2/session-3
m = csvread('eNTERFACE06_EMOBRAIN\Data\EEG\Part2_IAPS_SES3_EEG_fNIRS_09082006」
↳ .bdf.mrk.csv')
    case 3
%Participant-3/session-3
m = csvread('eNTERFACE06_EMOBRAIN\Data\EEG\Part3_IAPS_SES3_EEG_fNIRS_08082006」
↳ .bdf.mrk.csv')
    case 4
%Participant-4/session-3
m = csvread('eNTERFACE06_EMOBRAIN\Data\EEG\Part4_IAPS_SES3_EEG_fNIRS_09082006」
↳ .bdf.mrk.csv')
    case 5
%Participant-5/session-3
m = csvread('eNTERFACE06_EMOBRAIN\Data\EEG\Part5_IAPS_SES3_EEG_fNIRS_09082006」
↳ .bdf.mrk.csv')
end

```

```

%% reading class labels
labels_class
↪ =readtable('eINTERFACE06_EMOBRAIN\Data\Common\IAPS_Classes_EEG_fNIRS.csv')

%% PRE-PROCESSING
data=data'

%% taking chunks a different way
% Using the markers to assign labels by considering the label 1 as start of
% video and the following label as start of logic
no_of_rows_data= size(data,1) %No of total samples
no_of_columns_data= size(data,2)%No of electrodes

no_of_rows_m=size(m,1)%No of classes
no_of_columns_m=size(m,2)% No of markers related to each class

j=1
i=1
k=1
data_chunk_fromsynch={}
while i<= no_of_rows_data
    l= m(j,1)-1
    if i~=1
        data_chunk_fromsynch{k}= data(i:l,1:64)
        %data_chunk_fromsynch{k}= data(i:l,:)%Using All Columns
        k=k+1
    end
    i=m(j,1)
    j=j+2
    if j>no_of_rows_m
        break
    end
end

```

```

end

%% Annoting the data with labels
cl=zeros(30,1)
for i=1:1:30
    label_class=labels_class{i,3}
    if strcmp(label_class,'Calm')==1
        cl(i)=1; %CALM IS CLASS 1
    elseif strcmp(label_class,'Pos')==1
        cl(i)=2; %POSITIVE IS CLASS 2
    elseif strcmp(label_class,'Neg') ==1
        cl(i)=3;%NEGATIVE IS CLASS 3
    end
end

length(data_chunk_fromsynch)

%% feature extraction
%Zero meaning
zero_meaned_datachunk=zeromeaning(data_chunk_fromsynch)
clear data_chunk_fromsynch

%% Fourier transform
frequency_d_data = fourier_transform(zero_meaned_datachunk)

switch main_loop_counter
    case 1
        save ('participant1_session3_data_chunk','zero_meaned_datachunk')
    case 2
        save ('participant2_session3_data_chunk','zero_meaned_datachunk')
    case 3
        save ('participant3_session3_data_chunk','zero_meaned_datachunk')
    case 4
        save ('participant4_session3_data_chunk','zero_meaned_datachunk')
end

```

```

        case 5

            save ('participant5_session3_data_chunk','zero_meaned_datachunk')

        end

clear zero_meaned_datachunk

%% trying butterworth
% Wn=0.8
% [b,a] = butter(3, Wn, 'low');
% filteredSignal = filter(b, a, zero_meaned_data_f );
%filteredSignal = filteredSignal - mean(filteredSignal); % Subtracting the
↪ mean to block DC Component
%% Filter
%Keeping only alpha, beta, gamma,theta
% fcut = 60; %Hz
filtered_f_data = filtering_eeg(frequency_d_data,fs)
clear zero_meaned_datachunk
%% Getting time domain back
filtered_t_data=inverse_fourier(filtered_f_data)
switch main_loop_counter
    case 1
        save ('participant1_session3_filtered','filtered_f_data','filtered_t_」
↪ data')
    case 2
        save ('participant2_session3_filtered','filtered_f_data','filtered_t_」
↪ data')
    case 3
        save ('participant3_session3_filtered','filtered_f_data','filtered_t_」
↪ data')
    case 4
        save ('participant4_session3_filtered','filtered_f_data','filtered_t_」
↪ data')
    case 5

```

```

        save ('participant5_session3_filtered','filtered_f_data','filtered_t_」
        ↪ data')
end

clear filtered_f_data

%% zero meaning i.e remove DC
%zero_meaned_data = zeromeaning(filtered_t_data)

%% plotting zero meaned data
% figure
% hold on
% %plot(data_chunk{1}(:,1),'--')
% plot(zero_meaned_data{1}(:,5))
% hold off

%% Extracting statistical features time domain
% [mean_t, std_t] = statistical_feature_extraction(zero_meaned_data)

%% zeromeaning frequency data
%zero_meaned_data_f = zeromeaning(filtered_f_data)

%% Taking absolute value
%abs_value=absolute_value(zero_meaned_data_f)

%% Extracting frequency domain
%[mean_f, std_f] = statistical_feature_extraction(abs_value)

%% Consolidated to use in the ml app
% z=[mean_f  std_f mean_t std_t cl]
% z_t=[mean_t  std_t  cl]
% z_f=[mean_f  std_f  cl]

% %% PSD
% Frequencies=1024
% Fs=1024
% %Hpsd = dspdata.psd(Data,Frequencies)
% for i=1:1:length(filtered_f_data)
%     xdft=filtered_f_data{i}
%     N = length(xdft);

```

```

%      %xdft = fft(x);
% %      xdft = xdft(1:(N/2+1),:);
%      psdx1 = (1/(Fs*N)) * abs(xdft).^2;
%      psdx=sum(psdx1,1)
% %      Data =data_chunk_fromsynch{i}
% %      Frequencies=length(Data)
% %      Hpsd = dspdata.psd(abs(Data),Frequencies)
%      psd{i}=psdx
% %      psd{i}=Hpsd
% end

% %psdx(2:end-1) = 2*psdx(2:end-1);
%% Another way to calculate PSD
Fsx =1024
dx=1/Fsx

for i=1:1:length(filtered_t_data)
    x=filtered_t_data{i}
    L = length(x);
    Lfft = 2^nextpow2(L);
    dfx = 1/(Lfft*dx);
    fourier_image = fftshift(fft(x,Lfft)/Lfft);
    PSDfromFFT = abs(fourier_image).^2;
    freq = -Fsx/2:dfx:Fsx/2-dfx; % Frequency axis
    freqs{i}=freq
    PSD{i}=PSDfromFFT
end

clear x
clear L
clear Lfft

```

```

clear dfx
clear fourier_image
clear PSDfromFFT
clear freqs

%% Extracting features from power spectral density (frequency domain )
% [mean_p, std_p] = statistical_feature_extraction(abs_value)
% [max_p,min_p] =max_min(PSD)

%% whole set of statistical features
% features1 = extract(absolute_value(filtered_t_data))
% features2 = extract(absolute_value(filtered_f_data))
% features3 = extract(absolute_value(PSD))
%% Y
% trial=[features1 features2 features3 cl]
%% dwt
%   winsize:      window size (length of x)
%   wininc:       spacing of the windows (winsize)

for i=1:1:length(filtered_t_data)
    features_dw=[]
    for j=1:1:size(filtered_t_data{1},2)
        h= filtered_t_data{i}(:,j)
        winsize=length(h)
        SF=1024
        wininc=32
        feat = getmswtfeat(h,winsize,wininc,SF)
        features_dw=[features_dw feat]
    end
    features_discrete_wavelet(i,:)=features_dw
end

```

```

%% %% Discrete Wavelet Transform
% wname='db1' %Haar
% for i=1:1:length(filtered_f_data)
%     cA=[];
%     cH=[];
%     cV=[];
%     cD=[];
%
%     for j=1:1:size(filtered_f_data{1},2)
%         h= filtered_f_data{i}(:,j)
%         [ca,ch,cv,cd] = dwt2(h,wname)
%         cA = [cA ca]
%         cH=[cH ch]
%         cV=[cV cv]
%         cD=[cD cd]
%     end
%     CA{i}=[cA]
%     CH{i}=[cH]
%     CV{i}=[cV]
%     CD{i}=[cD]
%%         ca(i) = cA
%%         ch(i)=cH
%%         cv(i)= cV
%%         cd(i)= cD
% end

%% whole set of statistical features
features1 = extract(absolute_value(filtered_t_data))
features2 = extract(absolute_value(fourier_transform(filtered_t_data)))
features3 = extract(absolute_value(PSD))
features4 = More_extract(filtered_t_data)

```

```

features5 = More_extract(fourier_transform(filtered_t_data))
features6 = More_extract(PSD)
features7=Spectral_Features_Extract(filtered_t_data,SF)
features8=Spectral_Features_Extract(fourier_transform(filtered_t_data),SF)
features9=Spectral_Features_Extract(PSD,SF)
%features_discrete_wavelet
%features4 = extract(absolute_value(PSD))
% features=[features1 features2 features3 features_discrete_wavelet cl]
features=[features1 features2 features3 features4 features5 features6
↪ features7 features8 features9 features_discrete_wavelet cl]
switch main_loop_counter
    case 1
%save ('participant1_session3','features1','features2','features3','features_
↪ discrete_wavelet','features')
%save ('participant1_session3_more','features1','features2','features3','feat
↪ ures_discrete_wavelet','features')
save ('participant1_session3_4_13','features1','features2','features3','featu
↪ res4','features5','features6','features7','features8','features9','featur
↪ es_discrete_wavelet','features')
    case 2
% save ('participant2_session3','features1','features2','features3','features
↪ _discrete_wavelet','features')
%save ('participant2_session3_more','features1','features2','features3','feat
↪ ures_discrete_wavelet','features')
save ('participant2_session3_4_13','features1','features2','features3','featu
↪ res4','features5','features6','features7','features8','features9','featur
↪ es_discrete_wavelet','features')
    case 3
% save ('participant3_session3','features1','features2','features3','features
↪ _discrete_wavelet','features')
% save ('participant3_session3_more','features1','features2','features3','fea
↪ tures_discrete_wavelet','features')

```

```

save ('participant3_session3_4_13','features1','features2','features3','featu_
↪ res4','features5','features6','features7','features8','features9','featur_
↪ es_discrete_wavelet','features')

    case 4

% save ('participant4_session3','features1','features2','features3','features_
↪ _discrete_wavelet','features')

%save ('participant4_session3_more','features1','features2','features3','featu_
↪ ures_discrete_wavelet','features')

save ('participant4_session3_4_13','features1','features2','features3','featu_
↪ res4','features5','features6','features7','features8','features9','featur_
↪ es_discrete_wavelet','features')

    case 5

% save ('participant5_session3','features1','features2','features3','features_
↪ _discrete_wavelet','features')

%save ('participant5_session3_more','features1','features2','features3','featu_
↪ ures_discrete_wavelet','features')

save ('participant5_session3_4_13','features1','features2','features3','featu_
↪ res4','features5','features6','features7','features8','features9','featur_
↪ es_discrete_wavelet','features')

end

end

```

A.2 Feature Extraction Functions

A.2.1 Statistical Features

```

function features = extract(data)

l=size(data,2)

for i=1:1:l
    datachunk=data{i}
    %output_length = length(datachunk);
    output_mean = mean(datachunk,1);

```

```

output_SD = std(datachunk,1);
output_max = max(datachunk,[],1);
output_min = min(datachunk,[],1);
output_var=var(datachunk,1);
sk=skewness(datachunk,1);
k=kurtosis(datachunk,1);
mav=mad(datachunk,1);
%Simple Squared Integral
ssi = sum((datachunk).^2,1);
%Slope Sign change
for j=1:1:size(datachunk,2)
    summ=0;
    for counter = 2:1:length(datachunk)-1
        if ((datachunk(counter)-datachunk(counter-1))*(datachunk(counter)-datachunk(counter+1)))>0)
            a=1;
        else
            a=0;
        end
        summ = summ+a;
    end
    ssc(j)=summ;
end

% %Willison Amplitude
for j=1:1:size(datachunk,2)
    sum1=0
    for l = 1:1:(length(datachunk)-1)
        if (abs(datachunk(l)-datachunk(l+1)))>0)
            a=1;
        else
            a=0;

```

```

        end

        sum1 = sum1 +a;

    end

    wamp(j)=sum1

end

%log Detector

logan = exp(((1/length(datachunk)).*sum(log(abs(datachunk))))));

sum_first=zeros(1,size(datachunk,2));

sum_second=zeros(1,size(datachunk,2));

sum_first_norm=zeros(1,size(datachunk,2));

sum_second_norm=zeros(1,size(datachunk,2));

for len=1:1:(length(datachunk)-1)

    sum_first(1,:)=sum_first(1,:)+abs(datachunk(len+1,:)-datachunk(len,:))

    ↪ );

    sum_first_norm(1,:)=sum_first_norm(1,:)+abs(norm(datachunk(len+1,:))-

    ↪ norm(datachunk(len,:)));

end

for len=1:1:(length(datachunk)-2)

    sum_second(1,:)=sum_second(1,:)+abs(datachunk(len+2,:)-datachunk(len,

    ↪ :));

    sum_second_norm(1,:)=sum_second_norm(1,:)+abs(norm(datachunk(len+2,:))

    ↪ )-norm(datachunk(len,:)));

end

features(i,:) = [output_mean output_SD output_max output_min output_var

    ↪ sk k ssi ssc wamp mav output_var logan sum_first/(len-1)

    ↪ sum_first_norm/(len-1) sum_second/(len-2) sum_second_norm/(len-2)];

end

end

```

A.2.2 Spectral Features

```
function features = Spectral_Features_Extract(data,f)

% Peak amplitude | Generate a feature based on the amplitude of the peaks.
%
% Peak frequency | Generate a feature based on the frequency of the peaks.
%
% Peak value lower threshold | Constrain peak size to exclude low-amplitude
↳ peaks. For more information, in findpeaks, see the MinPeakHeight
↳ name-value pair argument.
%
% Number of peaks | Number of peaks to generate features for. The software
↳ selects N most prominent peaks in the chosen frequency band, going in the
↳ descending amplitude order. For more information, in findpeaks, see the
↳ NPeaks name-value pair argument.
%
% Minimum frequency gap | Specify a minimum frequency gap. If the gap between
↳ two peaks is less than this specification, the software ignores the
↳ smaller peak of the pair. For more information, in findpeaks, see the
↳ MinPeakDistance name-value pair argument.
%
% Peak excursion tolerance | Specify the minimum prominence of a peak. The
↳ prominence of a peak measures how much the peak stands out due to its
↳ intrinsic height and its location relative to other peaks. For more
↳ information, in findpeaks, see the MinPeakProminence name-value pair
↳ argument.

%Spectral moments

    %Spectral Centroid
    features=[]
    for len=1:1:length(data)
        X=data{len}
        %f=length(X)
```

```

%X = obw(x);
Spec_centroid(1,:) = mean(spectralCentroid(abs(X),f),1)
Spec_skewness(1,:) = mean(spectralSkewness(abs(X),f),1)
%sampled at rate or time interval sampx.
for column=1:1:size(X,2)
Spec_kurt(1,:) =mean(pkurtosis(abs(X(:,column)),f),1)
crest(1,:) = mean(spectralCrest(abs(X),f),1)
end

slope(1,:) = mean(spectralSlope(abs(X),f),1)
decrease(1,:) =mean(spectralDecrease(abs(X),f),1)
flux(1,:) = mean(spectralFlux(abs(X),f),1)

features(len,:)=[Spec_centroid Spec_skewness Spec_kurt crest slope
↪ decrease flux]
end
end

```

A.2.3 More Statistical Features and Event Related Potential

```

function features = More_extract(data)
s=[]
crr1=[]
crr2=[]
c=[]
for i=1:1:length(data)
X =data{i};
T='shannon';
E1(i) = wentropy(X,T);
T='log energy';
E2(i) = wentropy(X,T);
T='norm';
E3(i) = wentropy(X,T,1.1);

```

```

s =[s svd(X)]
%R2 = corrcoef(X)
%crr = xcorr2(X)
Corr2 =[]
Corr1=[]
c_dia=[]
for col = 1 : size(X,2)
    thisColumn = X(:, col);
    corr1=autocorr(thisColumn, 'NumLags', 1)
    % Call your custom-written function, autocorr().
    corr2 = autocorr(thisColumn, 'NumLags', 2)
    corr1=corr1'
    corr2=corr2'
    Corr2=[Corr2 corr2]
    Corr1=[Corr1 corr1]
    % Now do something with someOutput.
end
crr2 =[crr2;Corr2]
crr1 =[crr1;Corr1]
c=cov(X)
c_diagonal =diag(c)
c_diagonal=c_diagonal'
c_dia(i,:)=c_diagonal
end

s=s'
%ERP Features
p100(i,:)=max(X(81:123,:), [], 1)
n100(i,:)=min(X(81:123,:), [], 1)
p200(i,:)=max(X(153:282,:), [], 1)
n200(i,:)=max(X(153:282,:), [], 1)
p300(i,:)=max(X(281:512,:), [], 1)

```

```
features=[c_dia crr1 crr2 s E1' E2' E3' p100 n100 p200 n200 p300]
end
```

A.2.4 Discrete Wavelet Features

This code is borrowed and belongs to Rami Khushaba as cited.

```
% Copyright (c) 2017, Rami Khushaba
% All rights reserved.
%
% Redistribution and use in source and binary forms, with or without
% modification, are permitted provided that the following conditions are
% met:
%
% * Redistributions of source code must retain the above copyright
%   notice, this list of conditions and the following disclaimer.
% * Redistributions in binary form must reproduce the above copyright
%   notice, this list of conditions and the following disclaimer in
%   the documentation and/or other materials provided with the
%   ↪ distribution
%
% THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
% AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
% IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
% ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE
% LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
% CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
% SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
% INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
% CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
% ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
% POSSIBILITY OF SUCH DAMAGE.
```

```

%% Code starts below

% GETMSWTFEAT Gets the Multiscale Wavelet Transform features, these
% include: Energy, Variance, Standard Deviation, and Waveform Length
% feat = getmswtfeat(x,winsize,wininc,SF)
% -----

% The signals in x are divided into multiple windows of size
% "winsize" and the windows are spaced "wininc" apart.

% Inputs
% -----

%   x:                columns of signals
%   winsize:           window size (length of x)
%   wininc:            spacing of the windows (winsize)
%   SF:                sampling frequency (Not used in the current implementation,
%   ↪ but I left you some options down there)

% Outputs
% -----

%   feat:             WT features organized as [Energy, Variance, Waveform Length,
%   ↪ Entropy]

% Example
% -----

% feat = getmswtfeat(rand(1024,1),128,32,32)
% Assuming here rand(1024,1) (this can be any one dimensional signal,
% for example EEG or EMG) is a one dimensional signal sampled at 32
% for 32 seconds only. Utilizing a window size of 128 at 32 increments,
% features are extracted from the wavelet tree.
% I assumed 10 decomposition levels (J=10) below in the code.
% For a full tree at 10 levels you should get 11 features
% as we have decided to extract 4 types of features then we get 11 x 4 =44
% features.
% =====

```

```
% Multiscale Wavelet Transform feature extraction code by Dr. Rami Khushaba
% Research Fellow - Faculty of Engineering and IT
% University of Technology, Sydney.
% Email: Rami.Khushaba@uts.edu.au
% URL: www.rami-khushaba.com (Matlab Code Section)
% last modified 29/08/2012
% last modified 09/02/2013
```

```
function feat = getmswtfeat(x,winsize,wininc,SF)

if nargin < 4
    if nargin < 3
        if nargin < 2
            winsize = size(x,1);
        end
        wininc = winsize;
    end
    error('Please provide the sampling frequency of this signal')
end

datawin = ones(winsize,1);
datasize = size(x,1);
Nsignals = size(x,2);

%% Chop the signal according to a sliding window approach
numwin = floor((datasize - winsize)/wininc)+1;
% allocate memory
feat = zeros(winsize,numwin);
st = 1;
en = winsize;
for i = 1:numwin
    curwin = x(st:en,:).*repmat(datawin,1,Nsignals);
```

```

    feat(1:winsize,i) = detrend(curwin);

    st = st + wininc;
    en = en + wininc;
end

%% ----- Various options for J -----
% Note I put SF above in the inputs because you can use SF to determine the
% best decomposition level J, however for simplicity here I put it J=10;
J=10;% Number of decomposition levels which can also be set using
% or J=wmaxlev(winsize,'Sym5');
% or J=(log(SF/2)/log(2))-1;
%% Multisignal one-dimensional wavelet transform decomposition
dec = mdwtdec('col',feat,J,'db4');
% Proceed with Multisignal 1-D decomposition energy distribution

if isequal(dec.dirDec,'c')
    dim = 1;
end

[cfs,longs] = wdec2cl(dec,'all');
level = length(longs)-2;

if dim==1
    cfs = cfs';
    longs = longs';
end

numOfSIGs = size(cfs,1);
num_CFS_TOT = size(cfs,2);
absCFS = abs(cfs);
absCFS0 = (cfs);
cfs_POW2 = absCFS.^2;
Energy = sum(cfs_POW2,2);
percentENER = 0*ones(size(cfs_POW2));

```

```

notZER = (Energy>0);
percentENER(notZER,:) =
    ↪ 100*cfs_POW2(notZER,)./Energy(notZER,ones(1,num_CFS_TOT));

%% or try this version below and tell us which one is the best on your data
% percentENER(notZER,:) = cfs_POW2(notZER,:);

%% Pre-define and allocate memory
tab_ENER = zeros(numOfSIGs,level+1);
tab_VAR = zeros(numOfSIGs,level+1);
% tab_STD = zeros(numOfSIGs,level+1);
tab_WL = zeros(numOfSIGs,level+1);
tab_entropy = zeros(numOfSIGs,level+1);

%% Feature extraction section
st = 1;
for k=1:level+1
    nbCFS = longs(k);
    en = st+nbCFS-1;
    tab_ENER(:,k) =
        ↪ mean(percentENER(:,st:en),2);%.*sum(abs(diff(absCFS0(:,st:en)'))',2);
        ↪ % energy per waveform length
    tab_VAR(:,k) = var(percentENER(:,st:en),0,2); % variance of coefficients
    tab_STD(:,k) = std(percentENER(:,st:en),[],2); % standard deviation of
        ↪ coefficients
    tab_WL(:,k) = sum(abs(diff(percentENER(:,st:en)')).^2))'; % waveform length
    percentENER(:,st:en) = percentENER(:,st:en)./repmat(sum(percentENER(:,st:
        ↪ en),2),1,size(percentENER(:,st:en),2));
    tab_entropy(:,k) = -sum(percentENER(:,st:en).*log(percentENER(:,st:en)),2
        ↪ )./size(percentENER(:,st:en),2);
    st = en + 1;

```

```
end  
feat =([log1p([tab_ENER tab_VAR tab_WL]) tab_entropy]);
```

A.3 Saving Data

```
%Data = dlmread('Concrete_Data.csv', ',', 2, 0);  
x= load('participant3_session1_4_13')  
Data=x.features  
delete x  
x= load('participant4_session1_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant5_session1_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant1_session2_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant2_session2_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant3_session2_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant4_session2_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant5_session2_4_13')  
Data=[Data;x.features]  
delete x  
x= load('participant1_session3_4_13')  
Data=[Data;x.features]
```

```
delete x
x= load('participant2_session3_4_13')
Data=[Data;x.features]
delete x
x= load('participant3_session3_4_13')
Data=[Data;x.features]
delete x
x= load('participant4_session3_4_13')
Data=[Data;x.features]
delete x
x= load('participant5_session3_4_13')
Data=[Data;x.features]
delete x
%% Saving to use later
save ('All_participant_All_session_features_4_18','Data')
%% Saving again like
X=Data(:,1:size(Data,2)-1);
y=Data(:,size(Data,2));
save ('All_participant_All_sessions','X','y')

%% Save train and test split
XdataTrain = Data(1:360,1:size(Data,2)-1);
yTrain = Data(1:360,length(Data));
% xx=[XdataTrain yTrain]
XdataTest = Data(360:390,1:size(Data,2)-1);
yTest = Data(360:390,size(Data,2));
save('All_participants_train_test','XdataTrain','yTrain','XdataTest','yTest')
%% For classifier
X_classifier_train=abs(Data(1:360,:))
```

Appendix B

Python Code

B.1 Feature Reduction and Feature Selection

B.1.1 PCA

```
X=abs(data["XdataTrain"])
y=np.ravel(data["yTrain"])
X_test=abs(data["XdataTest"])
y_test=np.ravel(data["yTest"])
np.concatenate((X, X_test), axis=0)
np.concatenate((y, y_test), axis=0)
sc = StandardScaler() # creating a StandardScaler object
X_std = sc.fit_transform(X) # standardizing the data
pca = PCA()
X_pca = pca.fit(X_std)
pca = PCA(n_components = 0.99)
X_pca = pca.fit_transform(X_std) # this will fit and reduce dimensions
```

B.1.2 Random Forest for feature selection

```
# Get numerical feature importances
rf_exp = RandomForestClassifier(n_estimators= 9548, random_state=100)
rf_exp.fit(X, y)
```

```

importances = list(rf_exp.feature_importances_)# List of tuples with variable
↳ and importance
feature_list=list(range(1, 9548))
feature_importances = [(feature, round(importance, 2)) for feature,
↳ importance in zip(feature_list, importances)]# Sort the feature
↳ importances by most important first
feature_importances = sorted(feature_importances, key = lambda x: x[1],
↳ reverse = True)
# Make predictions on test data
X_test=abs(data["XdataTest"])
y_test=np.ravel(data["yTest"])
predictions = rf_exp.predict(X_test)# Performance metrics
errors = abs(predictions - y_test)

# Get numerical feature importances
importances = list(rf_exp.feature_importances_)# List of tuples with variable
↳ and importance
feature_importances = [(feature, round(importance, 2)) for feature,
↳ importance in zip(feature_list, importances)]# Sort the feature
↳ importances by most important first
feature_importances = sorted(feature_importances, key = lambda x: x[1],
↳ reverse = True)# Print out the feature and importances
[print('Variable: {:20} Importance: {}'.format(*pair)) for pair in
↳ feature_importances]

# list of x locations for plotting
import matplotlib.pyplot as plt

x_values = list(range(len(importances)))# Make a bar chart
plt.bar(x_values, importances, orientation = 'vertical', color = 'r',
↳ edgecolor = 'k', linewidth = 1.2)# Tick labels for x axis
plt.xticks(x_values, feature_list, rotation='vertical')# Axis labels and title

```

```
plt.ylabel('Importance'); plt.xlabel('Variable'); plt.title('Variable
↪ Importances');
```

B.2 Classification

A single SVM classifier

B.2.1 SVM

```
from sklearn.neighbors import KNeighborsClassifier
clf=KNeighborsClassifier()
clf.fit(X,y)
print("Training")
print(clf.score(X, y))
print("Testing")
print(clf.score(X_test, y_test))
```

An ensemble SVM

```
from sklearn.svm import SVC
from itertools import product
from sklearn.ensemble import VotingClassifier

# Training classifiers
clf1 = SVC(kernel='linear', probability=True)
clf2 = SVC(kernel='poly', probability=True)
clf3 = SVC(kernel='rbf', probability=True)
clf4 = SVC(kernel='sigmoid', probability=True)
ecf = VotingClassifier(estimators=[('svc', clf1), ('svc', clf2), ('svc',
↪ clf3), ('svc', clf4)],
                      voting='soft', weights=[2, 1, 2, 1])
```

```

clf1 = clf1.fit(X, y)
clf2 = clf2.fit(X, y)
clf3 = clf3.fit(X, y)
clf3 = clf3.fit(X, y)
eclf = eclf.fit(X, y)
print("Training")
print(clf.score(X, y))
print("Testing")
print(clf.score(X_test, y_test))

```

B.2.2 KNN

Ensemble KNN

```

from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
bagging = BaggingClassifier(KNeighborsClassifier(),
                           max_samples=0.8, max_features=0.8, oob_score=True,
                           ↪ , bootstrap=True, bootstrap_features=True)
bagging.fit(X_train, y_train)
print("Training")
print(bagging.score(X_train, y_train))
print("Testing")
bagging.score(X_test, y_test)

```

Single KNN

```

from sklearn.neighbors import KNeighborsClassifier
clf=KNeighborsClassifier()
clf.fit(X,y)
print("Training")
print(clf.score(X, y))
print("Testing")
print(clf.score(X_test, y_test))

```

B.2.3 Random Forest

Best Parameter Search

```

from mlxtend.feature_selection import SequentialFeatureSelector as SFS
from mlxtend.plotting import plot_sequential_feature_selection as plot_sfs
from sklearn.ensemble import RandomForestClassifier
from scipy.io import loadmat
import numpy as np
from sklearn.model_selection import RandomizedSearchCV
import sklearn
import matplotlib.pyplot as plt
from sklearn.datasets import load_boston
#boston = load_boston()
data =loadmat('D:\Final Year
↳ Project\eeg_read_bdf\All_participants_train_test')
X=abs(data["XdataTrain"])
#print(X)
y=np.ravel(data["yTrain"])
#print(sklearn.metrics.SCORERS.keys())

# Number of trees in random forest
n_estimators = [int(x) for x in np.linspace(start = 200, stop = 9548, num =
↳ 10)]

# Number of features to consider at every split
max_features = ['auto', 'sqrt']

# Maximum number of levels in tree
max_depth = [int(x) for x in np.linspace(10, 9548, num = 10)]
max_depth.append(None)

# Minimum number of samples required to split a node
min_samples_split = [2, 5, 10]

# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2, 4]

```

```

# Method of selecting samples for training each tree
bootstrap = [True, False]

# Create the random grid
random_grid = {'n_estimators': n_estimators,
               'max_features': max_features,
               'max_depth': max_depth,
               'min_samples_split': min_samples_split,
               'min_samples_leaf': min_samples_leaf,
               'bootstrap': bootstrap}

print(random_grid)

# Use the random grid to search for best hyperparameters
# First create the base model to tune
rf = RandomForestClassifier()

# Random search of parameters, using 3 fold cross validation,
# search across 100 different combinations, and use all available cores
rf_random = RandomizedSearchCV(estimator = rf, param_distributions =
    ↪ random_grid, n_iter = 100, cv = 10, verbose=2, random_state=42, n_jobs =
    ↪ -1)

# Fit the random search model
rf_random.fit(X, y)

print(rf_random.best_params_)
print(rf_random.best_score_)

lr = rf_random.best_estimator_

def evaluate(model, test_features, test_labels):
    predictions = model.predict(test_features)
    errors = abs(predictions - test_labels)
    mape = np.mean(errors / test_labels)
    accuracy = 100 - mape
    print('Model Performance')

```

```

    print('Average Error: {:.4f} degrees.'.format(np.mean(errors)))
    print('Accuracy = {:.2f}%.'.format(accuracy))

    return accuracy

test_features=abs(data["XdataTest"])
#print(X)
test_labels=np.ravel(data["yTest"])
best_random = rf_random.best_estimator_
random_accuracy = evaluate(best_random, test_features, test_labels)
#{'n_estimators': 1238, 'min_samples_split': 2, 'min_samples_leaf': 1,
→ 'max_features': 'sqrt', 'max_depth': 300, 'bootstrap': True}
#0.36666666666666667
'''{'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 2,
→ 'max_features': 'sqrt', 'max_depth': 4249, 'bootstrap': False}
0.386111111111111107
Model Performance
Average Error: 98.7742 degrees.
Accuracy = -3617.20%. '''

'''GRID SEARCH
{'bootstrap': True, 'max_depth': 4614, 'max_features': 'sqrt',
→ 'min_samples_leaf': 3, 'min_samples_split': 2, 'n_estimators': 100}
0.39166666666666666
Model Performance
Average Error: 123.4516 degrees.
Accuracy = 50.49%. '''
'''

# Number of trees in random forest
n_estimators = [int(x) for x in np.linspace(start = 100, stop = 500, num = 1)]
# Number of features to consider at every split
max_features = ['sqrt']

```

```
# Maximum number of levels in tree
max_depth = [int(x) for x in np.linspace(3000, 5000, num = 110)]
max_depth.append(None)

# Minimum number of samples required to split a node
min_samples_split = [2, 1, 10]

# Minimum number of samples required at each leaf node
min_samples_leaf = [1, 2, 3]

# Method of selecting samples for training each tree
bootstrap = [True]

from sklearn.model_selection import GridSearchCV

# Create the parameter grid based on the results of random search
param_grid = {
    'bootstrap': [True],
    'max_depth': max_depth,
    'max_features': max_features,
    'min_samples_leaf': min_samples_leaf,
    'min_samples_split': min_samples_split,
    'n_estimators': n_estimators
}

rf = RandomForestClassifier()

# Instantiate the grid search model
grid_search = GridSearchCV(estimator = rf, param_grid = param_grid,
                           cv = 3, n_jobs = -1, verbose = 2)

# Fit the grid search to the data
grid_search.fit(X, y)
print('GRID SEARCH')
print(grid_search.best_params_)
print(grid_search.best_score_)
best_grid = grid_search.best_estimator_
random_accuracy = evaluate(best_grid, test_features, test_labels)'''
'''
```

```

sfs = SFS(lr,
          k_features=9548,
          forward=True,
          floating=True,
          verbose=2,
          scoring='accuracy',
          cv=10)
sfs = sfs.fit(X, y)

fig = plot_sfs(sfs.get_metric_dict(), kind='accuracy')
plt.title('Sequential Forward Selection (w. accuracy)')
plt.grid()
plt.show()'''

```

B.2.4 Adaboost

```

clf = AdaBoostClassifier(n_estimators=9548, random_state=0)
clf.fit(X, y)
#AdaBoostClassifier(n_estimators=208, random_state=0)
#print(clf.feature_importances_)
print(clf.score(X, y))
X_test=abs(data["XdataTest"])
y_test=np.ravel(data["yTest"])
clf.score(X_test, y_test)

```

B.2.5 Neural Network

```

from keras.models import Sequential
from keras.layers import Dense, Dropout
#from keras.layers import BatchNormalization
from sklearn.model_selection import train_test_split
from keras.layers.normalization import BatchNormalization
from tensorflow.python.keras.utils import to_categorical
from keras import optimizers

```

```

from scipy.io import loadmat

import numpy as np

#create model
model = Sequential()

#loading data
data =loadmat('./All_participants_train_test.mat')
train_X=abs(data["XdataTrain"])
#print(X)
#train_y=np.ravel(data["yTrain"])
train_y=data["yTrain"]
print(train_y)
print(len(train_y))
test_X=abs(data["XdataTest"])
#get number of columns in training data
n_cols = train_X.shape[1]
n_rows = train_X.shape[0]
train_y=to_categorical(train_y)
test_y=data["yTest"]
test_y=to_categorical(test_y)
'''data =loadmat('./All_participant_All_sessions.mat')
print(data)
X=data["X"]
y=data["y"]
y=to_categorical(y)
n_cols = X.shape[1]
n_rows = X.shape[0]
train_X, test_X, train_y, test_y = train_test_split(X,y,
→ test_size=0.30,stratify = np.array(y),random_state = 100)'''

```

```
number_of_classes=4

#add model layers

model.add(Dense(n_cols,activation='relu', input_shape=(n_cols,)))
model.add(BatchNormalization())
#model.add(BatchNormalization())
#model.add(Activation('relu'))
model.add(Dropout(0.2))

model.add(Dense(round(n_cols/2), activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

model.add(Dense(round(n_cols/2), activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

'''model.add(Dense(round(n_cols/4), activation='relu'))
model.add(BatchNormalization())'''
#model.add(Dropout(0.2))

#model.add(Dense(1))#compile model using mse as a measure of model performance

model.add(Dense(number_of_classes, activation = 'softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy',metrics =
↳ ['accuracy'])
print(model.summary())

#from keras.callbacks import EarlyStopping
```

```
#set early stopping monitor so the model stops training when it won't improve
↳ anymore
#early_stopping_monitor = EarlyStopping(patience=3)
#train model
model.fit(train_X, train_y, validation_split=0.2, batch_size=36, epochs=300)
#example on how to use our newly trained model on how to make predictions on
↳ unseen data (we will pretend our new data is saved in a dataframe called
↳ 'test_X').
#test_y_predictions = model.predict(test_X)
score, accuracy = model.evaluate(test_X, test_y)
print('The testing accuracy is {}'.format(accuracy))
```